

МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра МО ЭВМ

Отчет
по лабораторной работе №4
по дисциплине «Введение в машинное обучение»
Тема: Кластеризация.

Студентка гр. 3384

Копасова К. А.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Цель работы: Изучить классификацию данных.

Задание: Вариант 34 - файлы lab4_roll.csv и lab4_blobs.csv.

1. Подготовка наборов данных:

1.1. Загрузите наборы.

1.2. Проверьте корректность загрузки.

1.3. Постройте диаграмму рассеяния набора данных. Опишите форму данных, и особенности в данных.

1.4. Подготовьте наборы данных проводя стандартизацию или нормировку данных. Обоснуйте выбор операции.

2. K-Means:

2.1. Проведите исследование оптимального количества кластеров методов локтя. Сделайте выводы, о наиболее подходящем количестве кластеров.

2.2. Проведите исследование оптимального количества кластеров методом силуэта. Сделайте выводы, о наиболее подходящем количестве кластеров.

2.3. Проведите кластеризацию алгоритмом K-means, с выбранным оптимальным количеством кластеров.

2.4. Постройте диаграмму рассеяния результатов кластеризации с выделением разным цветом разных кластеров.

2.5. Постройте диаграмму Вороного для результатов кластеризации. На диаграмме отметьте центроиды полученных кластеров.

2.6. Постройте для каждого признака диаграмму “box-plot” или “violin-plot”, с разделением по кластерам. Сделайте выводы о разделении кластеров и успешности применения кластеризации K-means к набору данных.

2.7. Рассчитайте для каждого кластера кол-во точек, среднее, СКО, минимум и максимум. Сопоставьте результаты с построенными графиками.

3. DBSCAN:

3.1. Подберите параметры алгоритма DBSCAN, которые на ваш взгляд дают наилучшие результаты. Опишите процесс (почему и как изменяли параметры) подбора параметров.

3.2. Постройте диаграмму рассеяния результатов кластеризации с выделением разным цветом разных кластеров.

3.3. Сделайте выводы об успешности кластеризации.

4. Иерархическая кластеризация:

4.1. Проведите иерархическую кластеризацию, подобрав параметр linkage. Постройте дендрограмму. Сделайте выводы, о разделении кластеров и необходимости изменить количество кластеров (если считаете, что необходимо изменить количество кластеров, то повторите кластеризацию с другим количеством кластеров).

4.2. Постройте диаграмму рассеяния результатов кластеризации с выделением разным цветом разных кластеров.

4.3. Сравните результаты кластеризации с результатами полученными в п.2 и п.3. Сделайте выводы о том, какой метод кластеризации подходит под каждый из наборов данных.

Выполнение работы

Выполним все задачи с файлом `lab4_roll.csv`.

1. Загрузка данных

1.1. Загрузка данных.

Загрузим данные из файла через функцию `read_csv()` библиотеки `pandas`.

```
import pandas as pd
data_roll = pd.read_csv('lab4_roll.csv')
print(data_roll.head())
```

Результат работы показан на рис. 1.1.

	Unnamed: 0	Weight	height
0	0	76.64	151.55
1	1	69.00	190.88
2	2	62.36	144.59
3	3	100.20	191.45
4	4	55.12	149.18

Рисунок 1.1 - Демонстрация набора данных.

Набор данных содержит следующие столбцы:

- 1) `Unnamed: 0` - индекс строки;
- 2) `Weight` - числовая характеристика, предположительно вес объекта;
- 3) `Height` - числовая характеристика, предположительно высота объекта.

Признаки `Weight` и `height` будут использоваться для группировки объектов в кластеры на основе их близости в пространстве признаков.

1.2. Проверка корректности загрузки.

Проверим данные на наличие пропущенных значений и корректность типов данных.

```
print(data_roll.describe())
print(data_roll.info())
```

Результат работы показан на рис. 1.2.1 и рис. 1.2.2.

	Unnamed: 0	Weight	height
count	1374.000000	1374.000000	1374.000000
mean	686.500000	77.916259	170.722271
std	396.783946	13.362495	20.848855
min	0.000000	52.320000	130.070000
25%	343.250000	65.700000	155.780000
50%	686.500000	80.800000	169.865000
75%	1029.750000	86.975000	187.632500
max	1373.000000	103.580000	216.500000

Рисунок 1.2.1 - Статистические характеристики набора данных.

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1374 entries, 0 to 1373
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Unnamed: 0   1374 non-null   int64
1   Weight       1374 non-null   float64
2   height       1374 non-null   float64
dtypes: float64(2), int64(1)
memory usage: 32.3 KB
None

```

Рисунок 1.2.2 - Информация о наборе данных.

В наборе содержится 1374 наблюдения, пропущенные значения отсутствуют во всех столбцах. Признаки Weight и height имеют вещественный тип (float64), что корректно для выполнения операций кластеризации.

Weight изменяется в пределах [52.32; 103.58], среднее значение ~ 77.92 , стандартное отклонение ~ 13.36 .

Height изменяется в пределах [130.07; 216.50], среднее значение ~ 170.72 , стандартное отклонение ~ 20.85 .

Признаки имеют разные диапазоны и стандартные отклонения (height варьируется сильнее, чем Weight). При использовании алгоритмов кластеризации, чувствительных к масштабу, признак с большим разбросом будет доминировать при расчете евклидова расстояния, поэтому нужно будет их масштабировать.

1.3. Построение диаграммы рассеяния набора данных.

Построим диаграмму рассеяния для визуальной оценки структуры данных и выявления потенциальных кластеров.

```

plt.scatter(data_roll['Weight'], data_roll['height'])
plt.xlabel('Weight')
plt.ylabel('height')
plt.title('Диаграмма рассеяния признаков')
plt.legend()
plt.show()

```

Результат работы показан на рис. 1.3.

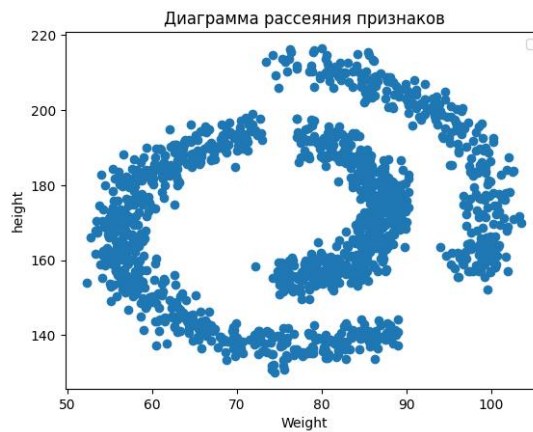


Рисунок 1.3 - Диаграмма рассеяния признаков.

На диаграмме рассеяния признаки Weight и height образуют несколько вытянутых изогнутых полос, напоминающих дуги или спираль.

Плотность распределения точек неравномерна, но визуально прослеживаются отдельные группы объектов, разделенные пустыми областями - это указывает на наличие естественных кластеров.

1.4. Подготовка набора данных.

Проверим распределение данных и наличие выбросов.

```
import matplotlib.pyplot as plt

plt.subplot(1,2,1)
plt.boxplot(data_roll['Weight'])
plt.title('Weight')

plt.subplot(1,2,2)
plt.boxplot(data_roll['height'])
plt.title('height')

plt.tight_layout()
plt.show()
```

Результат работы показан на рис. 1.4.1 и рис. 1.4.2.

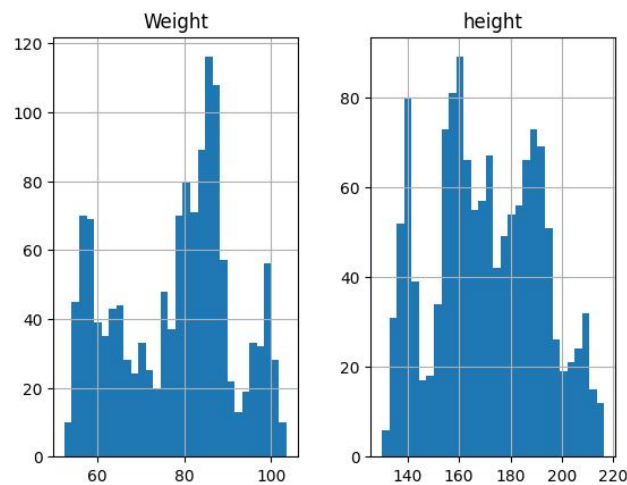


Рисунок 1.4.1 - Распределение признаков Weight и height.

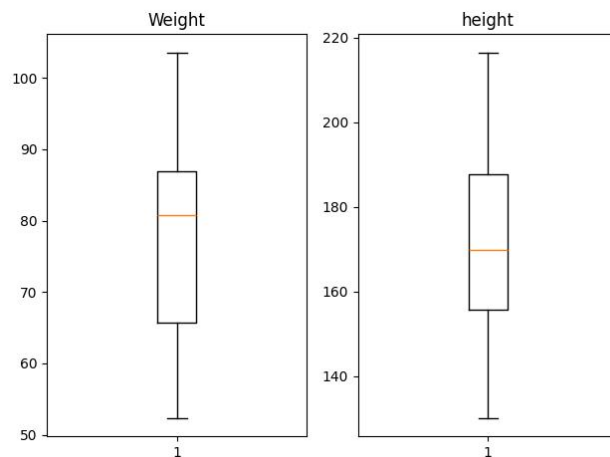


Рисунок 1.4.2 - Выбросы признаков Weight и height.

Распределение Weight имеет несколько пиков (55-60, 85-90 и 100), значения находятся в диапазоне от 52 до 104 - такая форма распределения говорит о том, что в данных предположительно есть несколько групп объектов с разным весом. Распределение height также имеет несколько пиков (140-145, 155-160, 175-180 и 190-195), значения находятся в диапазоне от 130 до 217 - это более широкое распределение по сравнению с признаком Weight.

Признаки не имеют выбросов, поэтому можно использовать стандартизацию StandardScaler - она сохранит форму распределения данных.

```
from sklearn.preprocessing import StandardScaler
```

```
X_role = data_roll[['Weight', 'height']]
print(X_role[:5])

scaler = StandardScaler()
X_role_scaled = scaler.fit_transform(X_role)
X_role_scaled_tf = pd.DataFrame(X_role_scaled, columns=['Weight', 'height'])
print('\n', X_role_scaled_tf.head())
```

Результат работы показан на рис. 1.4.3.

	Weight	height
0	-0.095545	-0.919919
1	-0.667503	0.967203
2	-1.164597	-1.253872
3	1.668240	0.994552
4	-1.706609	-1.033635

Рисунок 1.4.3 - Стандартизированные результаты.

После применения StandardScaler признаки приведены к единому масштабу.

Проверим получившиеся результаты через статистические характеристики и гистограмму распределения.

```
print(X_role_scaled_tf.describe())

X_role_scaled_tf.hist(bins=30)
plt.show()
```

Результат работы показан на рис. 1.4.4 и рис. 1.4.5.

	Weight	height
count	1.374000e+03	1.374000e+03
mean	3.354910e-16	7.356238e-16
std	1.000364e+00	1.000364e+00
min	-1.916227e+00	-1.950566e+00
25%	-9.145528e-01	-7.169560e-01
50%	2.158871e-01	-4.113333e-02
75%	6.781697e-01	8.113820e-01
max	1.921279e+00	2.196495e+00

Рисунок 1.4.4 - Статистические характеристики стандартизированных данных.

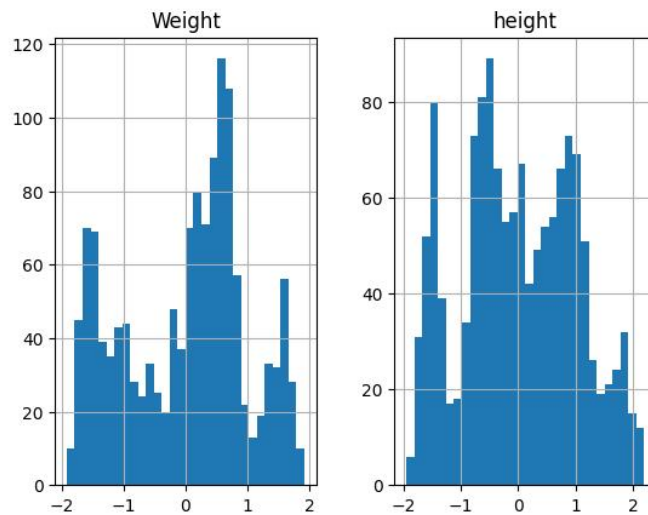


Рисунок 1.4.5 - Распределение стандартизированных признаков Weight и height.

Средние значения обоих признаков близко к 0, стандартное отклонение равно ~ 1 для каждого признака, диапазон значений - примерно от -2 до 2. Форма распределения сохранилась.

2. K-Means

2.1. Проведение исследования оптимального количества кластеров методом локтя.

Исследуем оптимальное количество кластеров для метода логтя.

```
from sklearn.cluster import KMeans

inertia = []

for k in range(1, 20):
    model = KMeans(n_clusters=k, random_state=42)
    model.fit(X_role_scaled_tf)
    inertia.append(model.inertia_)

plt.plot(range(1, 20), inertia, marker='o')
plt.title("Метод локтя")
plt.xlabel("k")
plt.xticks(range(1, 20))
plt.ylabel("Инерция")
plt.show()
```

Результат работы показан на рис. 2.1.

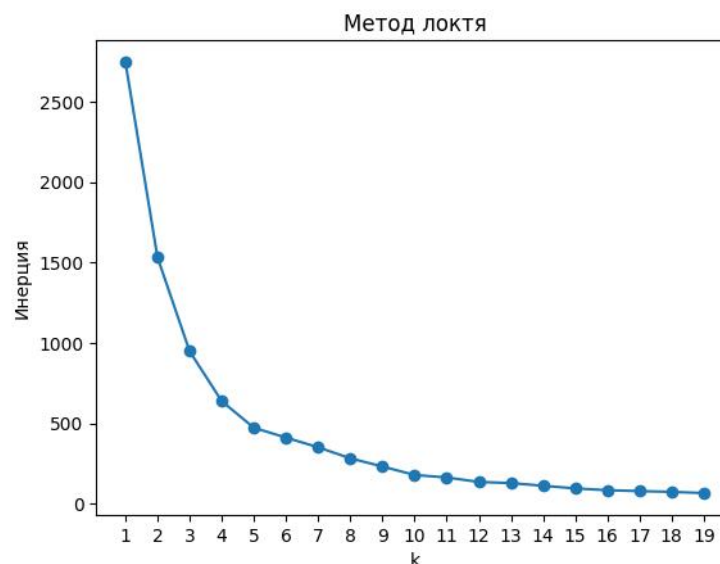


Рисунок 2.1 - Метод логтя.

Метод локтя показывает, как уменьшается ошибка кластеризации при увеличении количества кластеров. На графике видно, что при k от 1 до 3 инерция резко падает - это значит, что добавление кластеров сильно улучшает результат. После $k = 3-4$ кривая изгибается, и снижение замедляется - новые кластеры дают все меньший эффект. При $k > 6$ инерция плавно уменьшается до 100, что говорит о неэффективности дальнейшего увеличения количества кластеров. Оптимальное количество кластеров находится в диапазоне $k = 2-5$, где наблюдается точка перегиба графика.

2.2. Проведение исследования оптимального количества кластеров методом силуэта.

Исследуем оптимальное количество кластеров для метода силуэта.

```
from sklearn.metrics import silhouette_score

sil_scores = []

for k in range(2, 20):
    model = KMeans(n_clusters=k, random_state=42)
    labels = model.fit_predict(X_role_scaled_tf)
    sil_scores.append(silhouette_score(X_role_scaled_tf, labels))

plt.plot(range(2, 20), sil_scores, marker='o')
plt.title("Метод силуэта")
plt.xlabel("k")
plt.xticks(range(2, 20))
plt.ylabel("Коэффициент силуэта")
plt.show()
```

Результат работы показан на рис. 2.2.

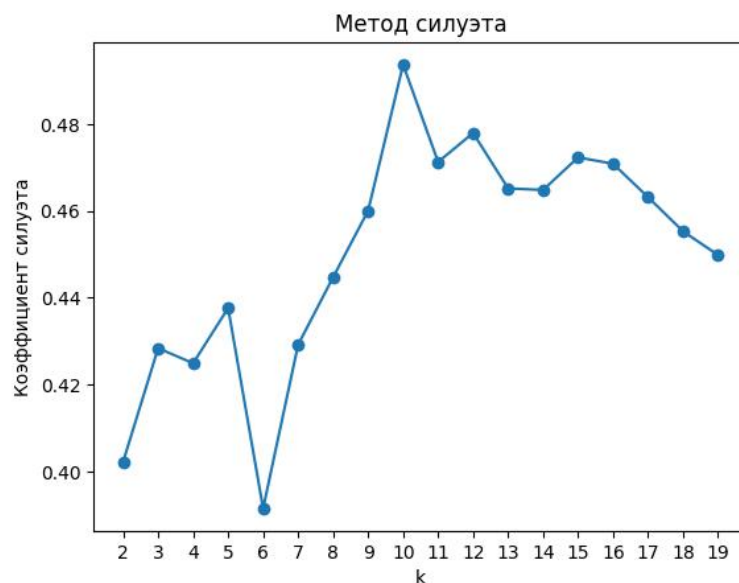


Рисунок 2.2 - Метод силуэта.

Метод силуэта оценивает качество кластеризации по тому, насколько хорошо каждый объект подходит к своему кластеру. Для каждой точки вычисляется коэффициент силуэта от -1 до 1 - он показывает, насколько объект похож на свой кластер по сравнению с другими кластерами (чем ближе к 1, тем лучше). На графике показан средний коэффициент силуэта для разного количества кластеров k . Максимальное значение >0.495 достигается при $k = 10$ - при таком количестве кластеров объекты лучше всего отделены друг от друга и компактны внутри своих кластеров. После $k = 10$ качество постепенно снижается, поэтому оптимальное количество кластеров - $k = 10$.

2.3. Проведение кластеризации алгоритмом K-means, с выбранным оптимальным количеством кластеров.

Поскольку метод локтя и метод силуэта показали совершенно разные результаты ($k=2-5$ и $k=10$ соответственно), то ради интереса исследуем разные значения количества кластеров и проверим разницу.

```
k_opt = [2, 3, 4, 5, 10]

for idx, k in enumerate(k_opt):
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X_role_scaled_tf)
    centroids = kmeans.cluster_centers_
```

Будем рассматривать 5 моделей кластеризации K-means.

2.4. Построение диаграммы рассеяния результатов кластеризации с выделением разным цветом разных кластеров.

Построим диаграммы рассеяния результатов.

```
k_opt = [2, 3, 4, 5, 10]

fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.ravel()

for idx, k in enumerate(k_opt):
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X_role_scaled_tf)
    centroids = kmeans.cluster_centers_

    scatter = axes[idx].scatter(X_role_scaled_tf['Weight'],
                                X_role_scaled_tf['height'], c=labels, cmap='tab10')
    axes[idx].scatter(centroids[:, 0], centroids[:, 1], c='red', marker='X',
                      s=200, edgecolors='black', label='Центроиды')
    axes[idx].set_title(f'K-Means, k={k}')
    axes[idx].set_xlabel('Weight')
    axes[idx].set_ylabel('height')
    axes[idx].legend()

    handles, _ = scatter.legend_elements(prop="colors", alpha=0.6)

    unique_labels = sorted(set(labels))
    labels_legend = []
    for label in unique_labels:
        labels_legend.append(f"Кластер {label}")

    centroid_marker = Line2D([0], [0], marker='X', color='w',
                              markerfacecolor='red', markersize=12, markeredgcolor='black', label='Центроиды')

    if len(labels_legend) <= 5:
        axes[idx].legend(handles=[*handles, centroid_marker],
                        labels=[*labels_legend, 'Центроиды'], fontsize=7, loc='best', framealpha=0.9)
    else:
        axes[idx].legend([centroid_marker], ['Центроиды'], fontsize=8,
                        loc='upper right', framealpha=0.9)
        axes[idx].text(0.02, 0.98, f'Кластеров: {k}',
                      transform=axes[idx].transAxes, fontsize=9, verticalalignment='top',
                      bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

axes[-1].axis('off')

plt.tight_layout()
plt.show()
```

Результат работы показан на рис. 2.4.

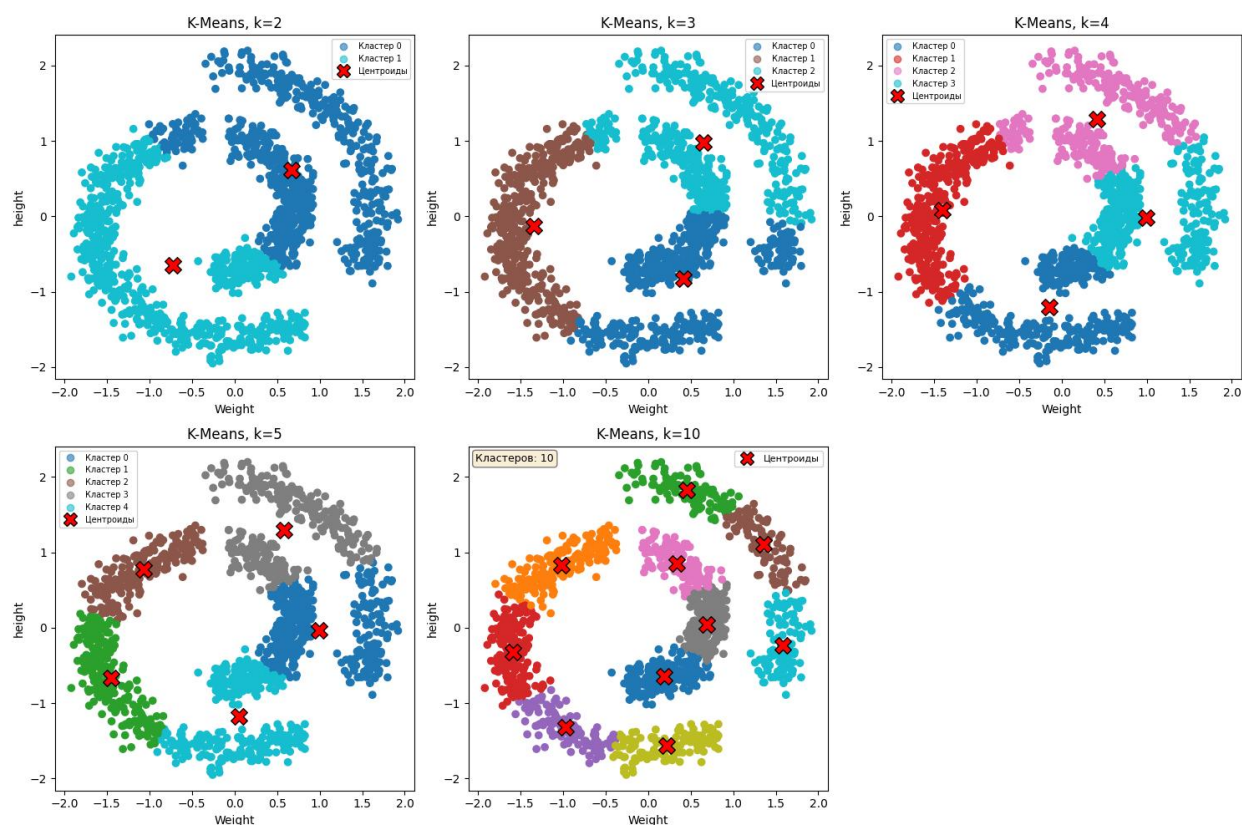


Рисунок 2.4 - Диаграммы рассеяния результатов кластеризации.

На графиках видно, что K-means не может выделить естественные кластеры в данных с изогнутой структурой. При всех значениях k алгоритм проводит линейные границы между кластерами, разделяя дуги поперек, вместо выделения их как целостных групп. При $k = 2$ данные делятся простой прямой линией, что не подходит для изогнутой структуры данных. При $k = 3-5$ кластеры становятся меньше, но продолжают разрезать изогнутые полосы. При $k = 10$ происходит избыточное дробление данных на мелкие части.

Визуально очевидно, что данные содержат три естественных группы, но K-means не подходит для данного набора данных. Алгоритм предполагает сферическую форму кластеров и не способен работать с нелинейно разделимыми структурами сложной формы.

2.5. Построение диаграммы Вороного для результатов кластеризации.

Диаграмма Вороного визуализирует границы областей влияния для каждого центроида. Она показывает, как K-means делит пространство признаков на зоны, где каждая точка принадлежит ближайшему центроиду.

Построим диаграммы Вороного для результатов кластеризации с классами $k = 3-5$ и $k = 10$. При $k = 2$ не удалось построить диаграмму Вороного, поскольку алгоритм Qhull требует больше двух точек центроидов для ее построения.

```
from scipy.spatial import Voronoi, voronoi_plot_2d
import numpy as np

fig, axes = plt.subplots(2, 2, figsize=(10, 10))
axes = axes.ravel()

k_opt = [3, 4, 5, 10]

for idx, k in enumerate(k_opt):
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X_role_scaled)
    centroids = kmeans.cluster_centers_

    centroids = np.unique(centroids, axis=0)

    vor = Voronoi(centroids)
    voronoi_plot_2d(vor, ax=axes[idx])

    scatter = axes[idx].scatter(X_role_scaled[:, 0], X_role_scaled[:, 1],
                                c=labels, cmap='tab10')

    axes[idx].scatter(centroids[:, 0], centroids[:, 1], c='red', marker='X',
                      s=200, edgecolors='black', zorder=5)

    axes[idx].set_title(f'K-Means, k={k}\nДиаграмма Вороного')
    axes[idx].set_xlabel('Weight')
    axes[idx].set_ylabel('height')
    axes[idx].grid(True, alpha=0.3)
    handles, _ = scatter.legend_elements(prop="colors", alpha=0.6)

    unique_labels = sorted(set(labels))
    labels_legend = []
    for label in unique_labels:
        labels_legend.append(f"Кластер {label}")

    centroid_marker = Line2D([0], [0], marker='X', color='w',
                              markerfacecolor='red', markersize=12, markeredgcolor='black', label='Центроиды')

    if len(labels_legend) <= 5:
        axes[idx].legend(handles=[*handles, centroid_marker],
                          labels=[*labels_legend, 'Центроиды'], fontsize=7, loc='best', framealpha=0.9)
    else:
        axes[idx].legend([centroid_marker], ['Центроиды'], fontsize=8,
                          loc='upper right', framealpha=0.9)
        axes[idx].text(0.02, 0.98, f'Кластеров: {k}',
                       transform=axes[idx].transAxes, fontsize=9, verticalalignment='top',
                       bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))
```

```
plt.tight_layout()
plt.show()
```

Результат работы показан на рис. 2.5.

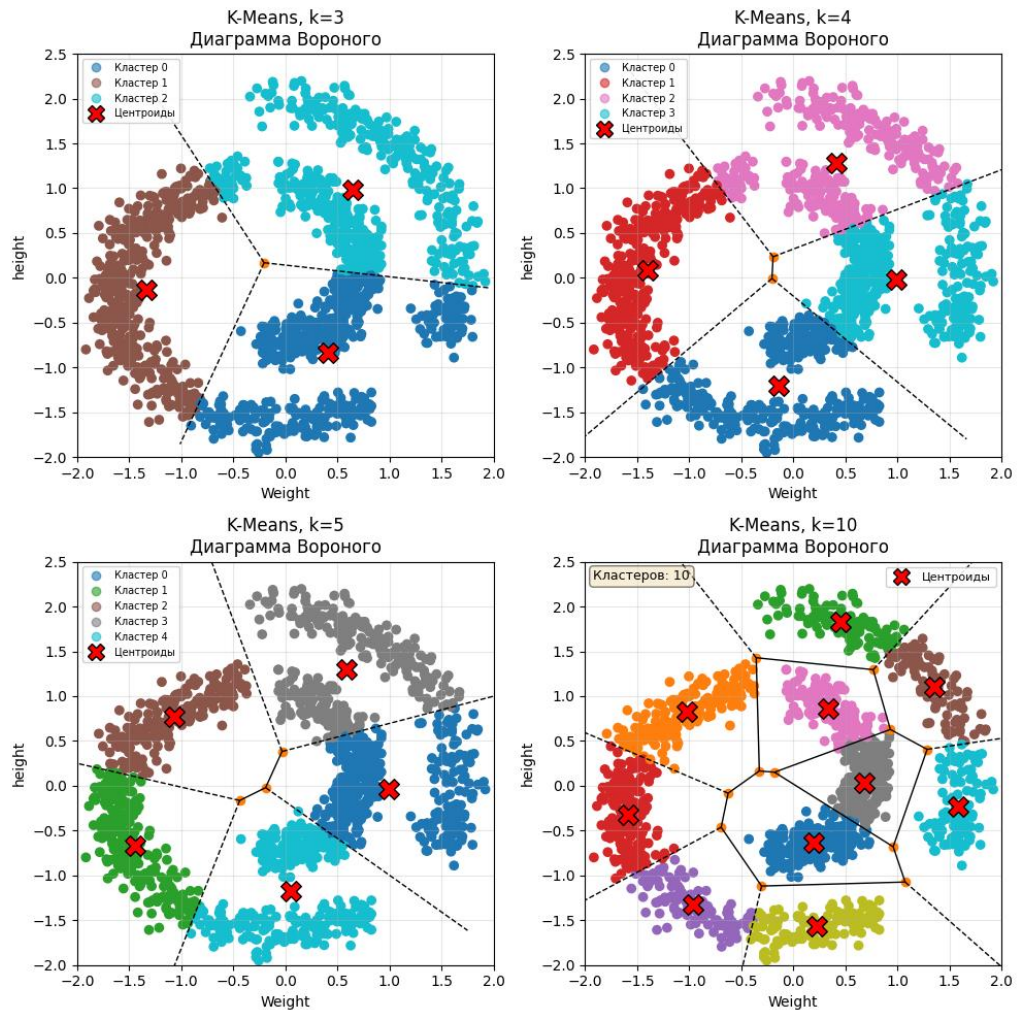


Рисунок 2.5 - Диаграммы Вороного для K-means при разном количестве кластеров.

На диаграммах Вороного видно разбиение пространства признаков на области влияния каждого центроида. Границы ячеек (серые линии) представляют собой линейные разделяющие поверхности между кластерами.

При $k = 3-4$ ячейки Вороного крупные и охватывают значительные области пространства. При $k = 5$ наблюдается более дробное разбиение. При $k = 10$ ячейки становятся мелкими, что приводит к избыточной фрагментации данных.

Полученные диаграммы подтверждают то, что K-means проводит между кластерами линейные границы, что не подходит для данных с изогнутой

нелинейной структурой. При увеличении k происходит излишнее дробление данных вместо выделения естественных кластеров. Для удобства дальнейшего детального анализа использую $k = 3$, т. к. это значение совпадает с количеством спиралей.

2.6. Выводы о разделении кластеров и успешности применения кластеризации K-means к набору данных.

Построим диаграммы box-plot для каждого признака для K-means с количеством кластеров $k = 3$.

```
import seaborn as sns

k_opt = 3
labels = kmeans_results[k_opt]['labels']

df_plot = X_role_scaled_tf.copy()
df_plot['cluster'] = labels
df_plot['cluster'] = df_plot['cluster'].astype(str)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.boxplot(x='cluster', y='Weight', data=df_plot, ax=axes[0])
axes[0].set_title('Weight по кластерам')
axes[0].set_xlabel('Кластер')
axes[0].set_ylabel('Weight (стандартизированный)')
axes[0].grid(alpha=0.3, axis='y')

sns.boxplot(x='cluster', y='height', data=df_plot, ax=axes[1])
axes[1].set_title('Height по кластерам')
axes[1].set_xlabel('Кластер')
axes[1].set_ylabel('Height (стандартизированный)')
axes[1].grid(alpha=0.3, axis='y')

plt.tight_layout()
plt.show()
```

Результат работы показан на рис. 2.6.

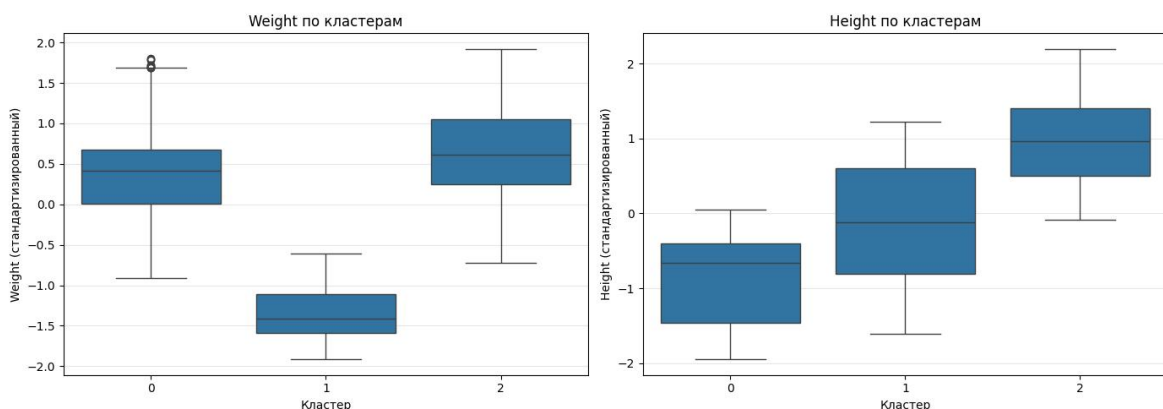


Рисунок 2.6 - Вох-plot для K-means кластеризации, $k=3$.

Диаграммы box-plot показывают значительное перекрытие распределений признаков между кластерами. По параметру Weight кластеры 0 и 2 имеют близкие медианы и интервалы - это указывает на слабую разделимость данных по признаку, кластер 0 имеет выбросы. По height классы 0 и 1, 1 и 2 также частично накладываются по диапазону значений - это может говорить о недостаточной выраженности разделяющих признаков. В целом, алгоритм K-means не способен корректно выделить спиральные структуры.

По результатам кластеризации алгоритмом K-means можно сделать вывод, что качество разделения данных зависит от формы распределения точек.

При наличии кластеров сложной формы (например, вытянутых или нелинейных структур), алгоритм K-means работает не эффективно, так как он основывается на евклидовом расстоянии и разбивает пространство на выпуклые области. В таких случаях границы кластеров могут проходить некорректно, разделяя естественные группы данных.

2.7. Расчет количества точек, среднее, СКО, минимум и максимум. Сопоставление результатов с построенными графиками.

Рассчитаем количество точек, среднее, СКО, минимум и максимум для модели K-means k=3.

```
df_temp = X_role_scaled_tf.copy()
df_temp['cluster'] = kmeans_results[k_opt]['labels']

stats = df_temp.groupby('cluster').agg({
    'Weight': ['count', 'mean', 'std', 'min', 'max'],
    'height': ['count', 'mean', 'std', 'min', 'max']
})

print(stats)
```

Результат работы показан на рис. 2.7.

cluster	Weight					height				
	count	mean	std	min	max	count	mean	std	min	max
0	504	0.406575	0.597008	-0.905569	1.798503	504	-0.835499	0.562757	-1.950566	0.054590
1	387	-1.340188	0.297578	-1.916227	-0.607612	387	-0.128789	0.784261	-1.607977	1.220546
2	483	0.649563	0.608280	-0.724399	1.921279	483	0.975017	0.575932	-0.087916	2.196495

Рисунок 2.7 - Статистические характеристики признаков K-means для k=3.

По признаку Weight кластер 1 отделен от остальных - имеет отрицательное среднее (-1.34) и очень малое стандартное отклонение (0.30), что указывает на компактную группу с низким весом. Кластеры 0 и 2 имеют близкие положительные средние значения (0.41 и 0.65) и сопоставимый разброс (~ 0.60), их диапазоны сильно перекрываются (от -0.9 до 1.9), что говорит о плохом разделении этих групп по данному признаку.

По признаку height все три кластера имеют различные средние значения: кластер 0 - отрицательное (-0.84), кластер 1 - около нуля (-0.13), кластер 2 - положительное (0.98). При этом кластер 1 имеет наибольшее стандартное отклонение (0.78) и самый широкий диапазон (-1.61 до 1.22), что указывает на значительный разброс точек внутри этой группы. Кластеры 0 и 2 более компактны (std ~ 0.57), но их диапазоны частично пересекаются, особенно в области около нуля.

Ни один из признаков по отдельности не обеспечивает идеального разделения всех трех кластеров - наблюдается перекрытие распределений, что подтверждает вывод о некорректной работе K-means на спиральных данных: алгоритм разрезает естественные структуры, а не выделяет их как целостные группы.

3. DBSCAN

3.1. Подбор параметров алгоритма DBSCAN.

У алгоритма DBSCAN есть два основных параметра: `eps` - максимальное расстояние между двумя точками, чтобы считаться соседями и `min_samples` - минимальное количество точек для формирования кластера.

Сначала найдем нужный диапазон для `eps` - для этого нужно построить графики k-расстояний, где `k` соответствует параметру `min_samples`. Для параметра `min_samples` буду рассматривать диапазон от 2 до 10.

```
from sklearn.neighbors import NearestNeighbors

for k in range(2, 11):
    neigh = NearestNeighbors(n_neighbors=k)
    neigh.fit(X_role_scaled_tf)

    distances, _ = neigh.kneighbors(X_role_scaled_tf)
```

```

distances = np.sort(distances[:, -1])

plt.plot(distances, label=f'k={k}')

plt.xlabel('Точки (отсортированные по расстоянию)')
plt.ylabel(f'Расстояние до k-го соседа')
plt.title('График k-расстояний для подбора eps')
plt.legend(loc='best', fontsize=9)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```

Результат работы показан на рис. 3.1.1.

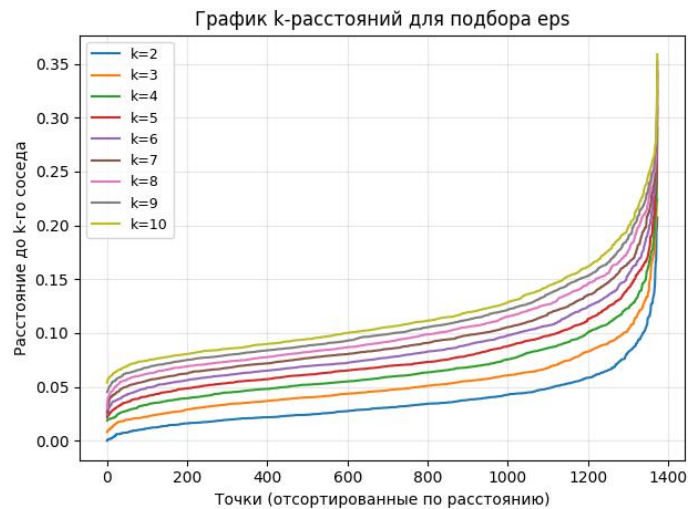


Рисунок 3.1.1 - График k-расстояний для подбора eps.

На диаграммах видно, что все графики имеют схожую изогнутую форму. Кривые имеют плавный участок до $x = 1200$, где расстояния между точками небольшие (0.05-0.15) - вероятно это точки, которые находятся внутри плотных областей данных. После $x = 1200$ наблюдается резкий рост кривых (0.25-0.35) - это шумовые точки или точки на границах кластерах.

На основе графика для детального исследования для eps был выбран диапазон от 0.05 до 0.35.

Кластеризируем данные с помощью DBSCAN при разных гиперпараметрах и построим зависимости количества кластеров от min_samples (рассмотрим также диапазон 2-10) для разных значений eps (0.05-0.35).

```

from sklearn.cluster import DBSCAN

eps_values = np.arange(0.05, 0.35, 0.03)
results = {}
min_samples_values = list(range(2, 10))

```

```

from sklearn.cluster import DBSCAN

eps_values = np.arange(0.1, 0.26, 0.03)
results = {}
min_samples_values = list(range(2, 16))

for min_samp in min_samples_values:
    results[min_samp] = {}
    for eps in eps_values:
        db = DBSCAN(eps=eps, min_samples=min_samp)
        labels = db.fit_predict(X_role_scaled_tf)

        n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
        noise = list(labels).count(-1)
        results[min_samp][eps] = {'clusters': n_clusters, 'noise': noise}

fig, axes = plt.subplots(2, 3, figsize=(20, 10))
axes = axes.ravel()

for idx, eps in enumerate(eps_values):
    cluster_values = [results[min_s][eps]['clusters'] for min_s in
min_samples_values]

    axes[idx].plot(min_samples_values, cluster_values, marker='o', linewidth=2,
markersize=6)

    axes[idx].axhline(y=3, color='green', linestyle='--', linewidth=2,
label='Ожидаемое количество кластеров (3)')
    axes[idx].set_xlabel('min_samples', fontsize=11)
    axes[idx].set_ylabel('Количество кластеров', fontsize=11)
    axes[idx].set_title(f'eps = {eps:.2f}', fontsize=12, fontweight='bold')
    axes[idx].grid(alpha=0.3)
    axes[idx].set_xticks(min_samples_values)
    axes[idx].set_ylim(0, max(cluster_values) + 1)
    axes[idx].legend(fontsize=9)

plt.suptitle('Зависимость количества кластеров от min_samples для разных
значений eps', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

```

Результат работы показан на рис. 3.1.2.

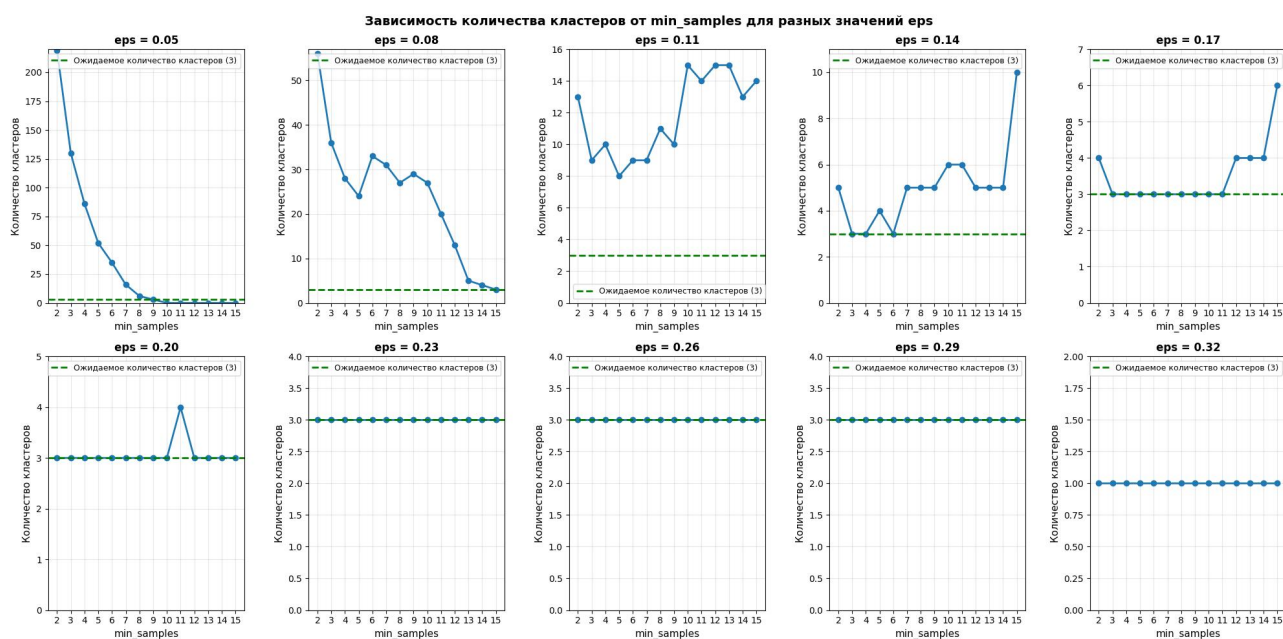


Рисунок 3.1.2 - Зависимость количества кластеров от min_samples для разных значений eps.

При малых eps (0.05-0.08) алгоритм выявляет сотни кластеров, что свидетельствует о чрезмерной фрагментации данных. По мере увеличения eps (0.11-0.17) количество кластеров снижается, но остается нестабильным и волнистым. При eps = 0.20-0.29 кривые становятся практически горизонтальными и стабильно определяют три кластера, что соответствует ожидаемому результату и демонстрирует минимальную зависимость от min_samples. При eps = 0.32 происходит полное объединение данных в один кластер. Оптимальный диапазон eps находится в интервале 0.20-0.29, где алгоритм показывает стабильные и надежные результаты.

Проанализирую значения min_samples = 2-4, поскольку они не усложняют модель и вычисления, при этом эти параметры достигают необходимую точность кластеризации, выявляя ровно три целевых кластера.

3.2. Построение диаграммы рассеяния результатов кластеризации с выделением разным цветом разных кластеров.

Построим диаграммы рассеяния для min_samples=2-4 и eps=0.05-0.32.

```
fig, axes = plt.subplots(2, 5, figsize=(15, 10))
axes = axes.ravel()

for i, eps in enumerate(eps_values):
    db = DBSCAN(eps=eps, min_samples=2)
    labels = db.fit_predict(X_role_scaled)

    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    n_noise = list(labels).count(-1)

    scatter = axes[i].scatter(X_role_scaled[:, 0], X_role_scaled[:, 1],
                              c=labels, cmap='tab10')

    axes[i].set_title(f'eps = {eps:.2f}\nКластеров: {n_clusters}, Шум: {n_noise}')
    axes[i].grid(alpha=0.3)

    handles, _ = scatter.legend_elements(prop="colors", alpha=0.6)

    unique_labels = sorted(set(labels))
    labels_legend = []
    for label in unique_labels:
        if label == -1:
            labels_legend.append("Шум")
```

```

else:
    labels_legend.append(f"Кластер {label}")

if len(labels_legend) <= 10:
    axes[i].legend(handles, labels_legend, fontsize=7, loc='best')
else:
    axes[i].legend([f"Всего кластеров: {n_clusters}"], fontsize=8,
loc='upper right')

plt.tight_layout()
plt.show()

```

Результат работы показан на рис. 3.2.1, рис. 3.2.2 и рис. 3.2.3.

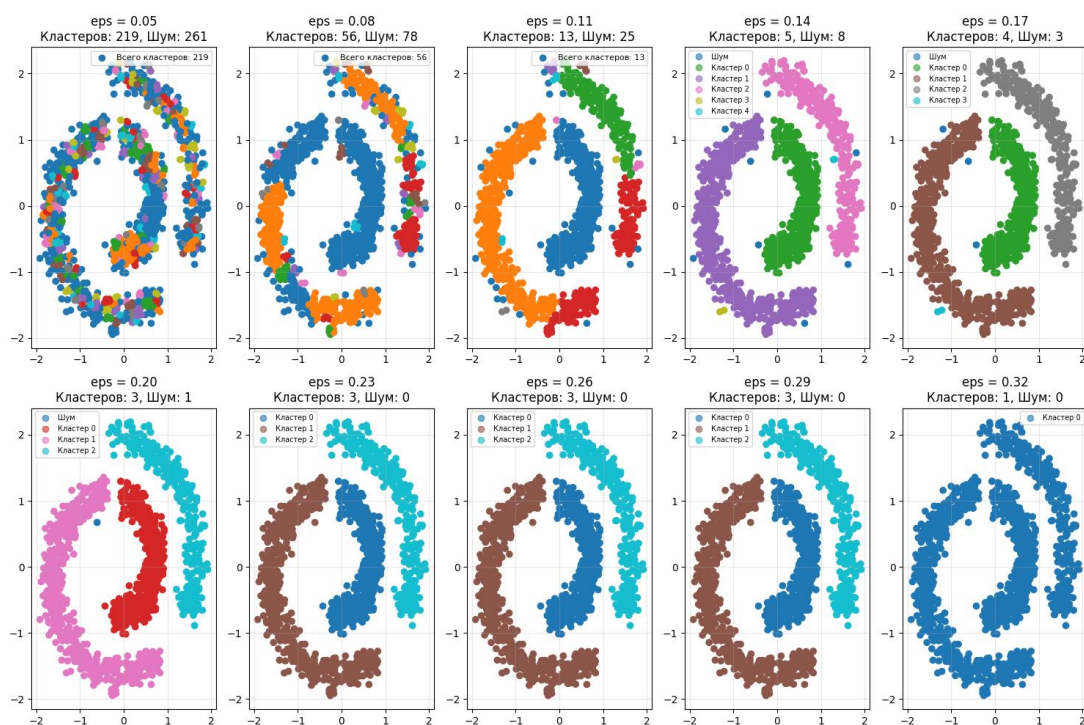


Рисунок 3.2.1 - Диаграммы рассеяния для разных eps и min_samples=2.

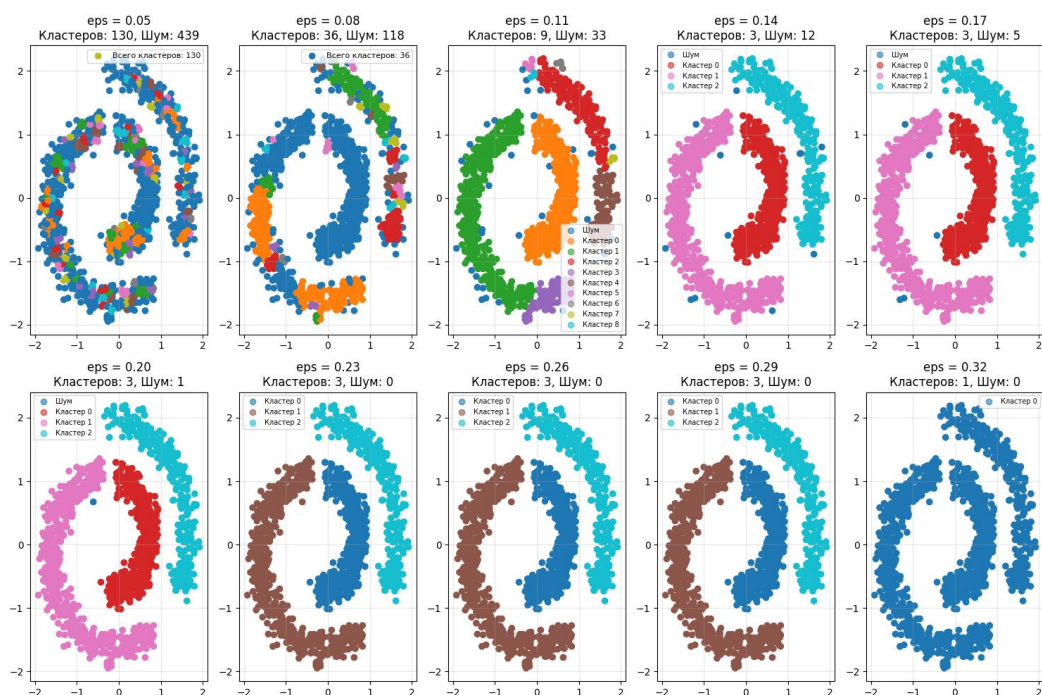


Рисунок 3.2.2 - Диаграммы рассеяния для разных eps и min_samples=3.

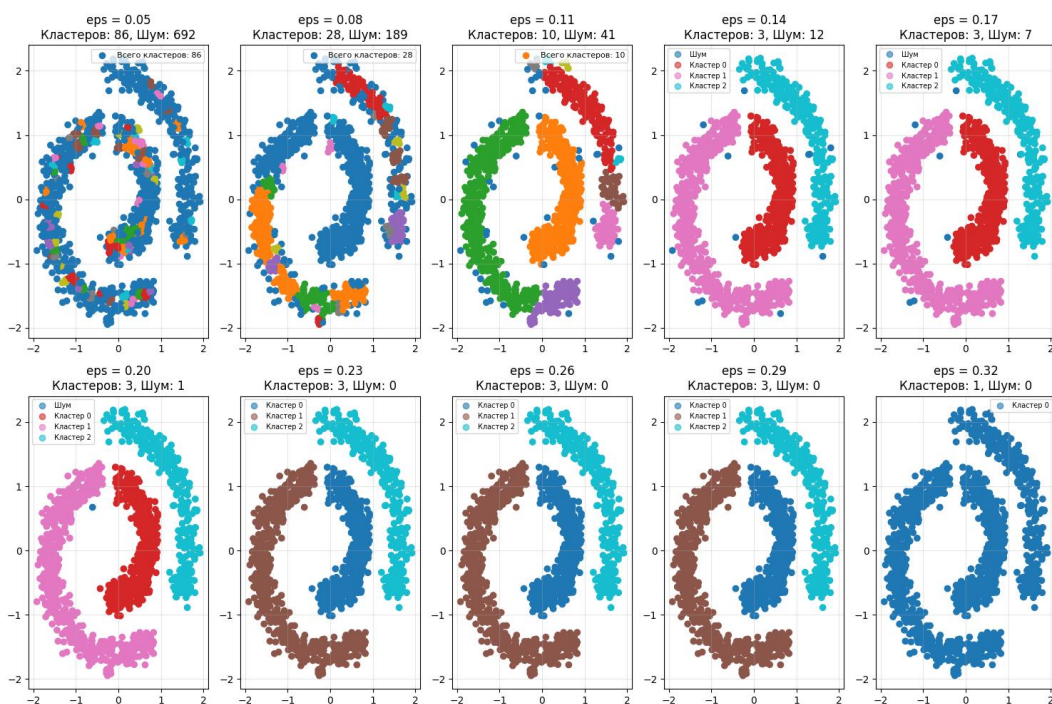


Рисунок 3.2.3 - Диаграммы рассеяния для разных eps и min_samples=4.

При малых eps (0.05-0.11) алгоритм выделяет много мелких кластеров (от 10 до 200+), разбивая спиральные структуры на фрагменты. Также при увеличении параметра min_samples наблюдается рост количества шумовых точек и

уменьшение количества кластеров, так как для вхождения в кластер точке требуется больше соседей.

При средних ϵ (0.14-0.17) виден переход к естественной структуре данных: выделяется 3-5 кластеров. Количество шума снижается до 3-12 точек, модель лучше выделяет группы. При этом чем больше параметр min_samples , тем быстрее выделяется нужное число кластеров, но растет доля шума.

При $\epsilon=0.20-0.29$ алгоритм устойчиво выделяет 3 кластера для всех значений min_samples . Количество шума минимально (0-1 точка) и границы между спиралями визуально четкие.

При большом $\epsilon=0.32$ все три спирали сливаются в один кластер вне зависимости от параметра min_samples , что говорит о потере структуры данных.

При $\text{min_samples}=2$ шум минимален, но модель более чувствительна к случайным скоплениям точек и может определить их как кластер (хотя кластером они могут и не являться). При таком значении параметра следует выбрать более высокий ϵ (0.20-0.23) для предотвращения лишних кластеров.

При $\text{min_samples}=3-4$ шум незначительно растет, но модель становится устойчивее к выбросам. При таком значении параметра нужно выбирать средний ϵ (0.14-0.23), чтобы не было переобучения.

3.3. Выводы об успешности кластеризации.

Эксперименты с DBSCAN показали, что алгоритм успешно справился с кластеризацией спирального набора данных, показав значительное преимущество перед методом K-means.

При оптимальных параметрах ($\text{min_samples}=2$ и $\epsilon=0.20-0.23$ или $\text{min_samples}=3-4$ и $\epsilon=0.14-0.23$) алгоритм выделил 3 основных кластера, что соответствует естественной структуре данных (три спиральные ветви). Количество шумовых точек минимальное (0-7), границы между кластерами визуально четкие, без смешивания точек из разных спиралей.

Таким образом, алгоритм DBSCAN применяется для данных с нелинейной структурой и плотностным распределением кластеров. Однако для корректной

работы требуется тщательный подбор гиперпараметров, в частности параметра `eps`, который определяет масштаб плотности соседства.

4. Иерархическая кластеризация

4.1. Проведение иерархической кластеризации, подобрав параметр `linkage`.

Построение дендрограммы.

Дендрограмма - это древовидная диаграмма, которая показывает, какие точки объединяются в кластеры на разных уровнях похожести.

Все методы иерархической кластеризации различаются способом измерения расстояния между кластерами. Параметр `linkage` может принимать следующие значения:

1) `Single Linkage` (метод ближайшего соседа) - расстояние между двумя кластерами равно минимальному расстоянию между любыми двумя точками из этих кластеров.

2) `Complete Linkage` (метод дальнего соседа) - расстояние между кластерами равно максимальному расстоянию между любыми двумя точками из этих кластеров.

3) `Average Linkage` (метод среднего расстояния) - расстояние между кластерами равно среднему расстоянию между всеми парами точек из разных кластеров.

4) `Ward Linkage` (метод Уорда) - этот метод объединяет два кластера, которые минимально увеличат внутрикластерную дисперсию (сумму квадратов отклонений точек от центроида кластера).

Построим для всех перечисленных методов `linkage` дендрограммы.

```
from scipy.cluster.hierarchy import dendrogram, linkage

linkage_methods = ['single', 'complete', 'average', 'ward']

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.ravel()
for idx, method in enumerate(linkage_methods):
    try:
        Z = linkage(X_role_scaled_tf, method=method)
        dendro_data = dendrogram(Z, no_plot=True)

        dendrogram(Z, ax=axes[idx], truncate_mode='lastp', p=30)
        axes[idx].set_title(f'Метод linkage: {method}')
        axes[idx].set_xlabel('Индекс точки')
```

```

        axes[idx].set_ylabel('Расстояние')
        axes[idx].grid(alpha=0.3)
    except Exception as e:
        axes[idx].text(0.5, 0.5, f'Ошибка: \n{str(e)}', ha='center',
va='center', transform=axes[idx].transAxes)
        axes[idx].set_title(f'Метод: {method}')

plt.suptitle('Дендрограммы для разных методов linkage', fontsize=14,
fontweight='bold')
plt.tight_layout()
plt.show()

```

Результат работы показан на рис. 4.1.

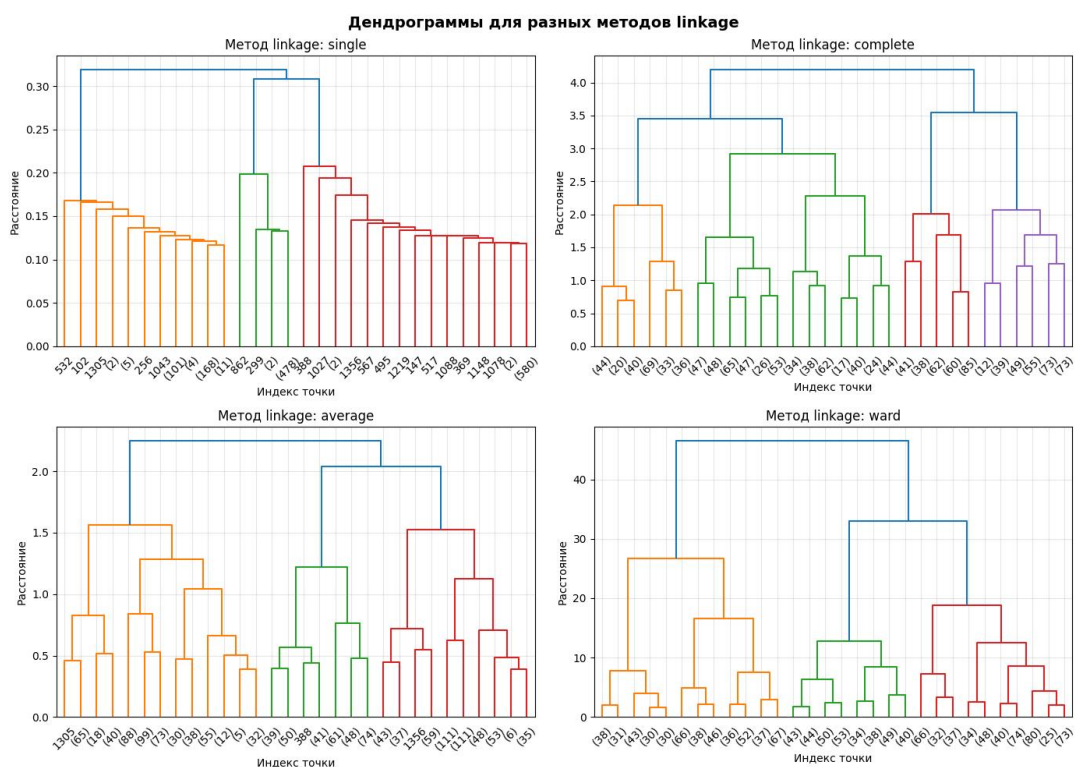


Рисунок 4.1 - Дендрограммы для разных методов linkage.

На дендрограмме метода single linkage прослеживаются три независимые ветви (оранжевая, зеленая и красная), которые развиваются параллельно на расстояниях 0.00-0.20 и сливаются в один кластер только на большом скачке (>0.30). Алгоритм выделил три основных группы.

На дендрограмме метода complete linkage прослеживается фрагментированную структуру с четырьмя основными ветвями (оранжевая, зеленая, красная и фиолетовая), которые сливаются на расстояниях 3.0-4.0. Метод разбивает спиральные структуры на множество мелких компактных групп.

На дендрограмме метода average linkage прослеживается три основные ветви (оранжевая, зеленая и красная), формирующиеся на расстояниях 0.5-1.5 и сливающиеся на уровне >2.0. Структура более сбалансирована по сравнению с complete linkage, однако слияние происходит на меньших расстояниях, что говорит о менее четком разделении кластеров.

Дендрограмма метода ward linkage показывает наибольшее расстояние слияния (после 30 единиц), что показывает стремление метода минимизировать внутрикластерную дисперсию. Три основные ветви видны отчетливо.

Поскольку три метода linkage (single, average и ward) выделили три основных кластера, что соответствует структуре данных, построим диаграммы рассеяния для каждого из трех методов.

4.2. Построение диаграммы рассеяния результатов кластеризации с выделением разным цветом разных кластеров.

Построим три диаграммы рассеяния для методов single, average и ward.

Метод single:

```
from sklearn.cluster import AgglomerativeClustering

hc = AgglomerativeClustering(n_clusters=3, linkage='single')
labels = hc.fit_predict(X_role_scaled_tf)

plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_role_scaled_tf.iloc[:, 0], X_role_scaled_tf.iloc[:, 1],
c=labels, cmap='tab10')

handles, _ = scatter.legend_elements(prop="colors", alpha=0.6)
labels_legend = [f"Кластер {i}" for i in range(3)]
plt.legend(handles, labels_legend, fontsize=9, loc='best')

plt.title(f'Диаграмма рассеяния (single), k=3')
plt.xlabel('Weight')
plt.ylabel('height')
plt.grid(alpha=0.3)
plt.show()
```

Результат работы показан на рис. 4.2.1.

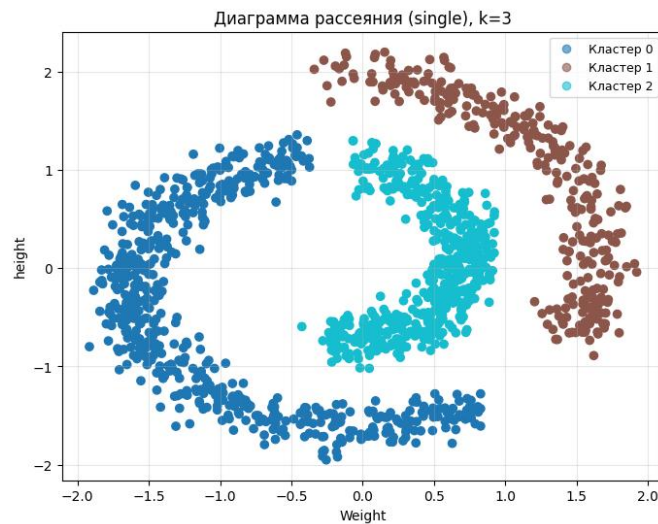


Рисунок 4.2.1 - Диаграмма рассеяния (single), k=3.

Алгоритм корректно выделил три спиральные структуры - каждый кластер окрашен в отдельный цвет. Границы между кластерами четкие, без смешивания точек из разных спиралей.

Метод average :

```
hc = AgglomerativeClustering(n_clusters=3, linkage='average')
labels = hc.fit_predict(X_role_scaled_tf)

plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_role_scaled_tf.iloc[:, 0], X_role_scaled_tf.iloc[:, 1],
c=labels, cmap='tab10')

handles, _ = scatter.legend_elements(prop="colors", alpha=0.6)
labels_legend = [f"Кластер {i}" for i in range(3)]
plt.legend(handles, labels_legend, fontsize=9, loc='best')
plt.title(f'Диаграмма рассеяния (average), k=3')
plt.xlabel('Weight')
plt.ylabel('height')
plt.grid(alpha=0.3)
plt.show()
```

Результат работы показан на рис. 4.2.2.

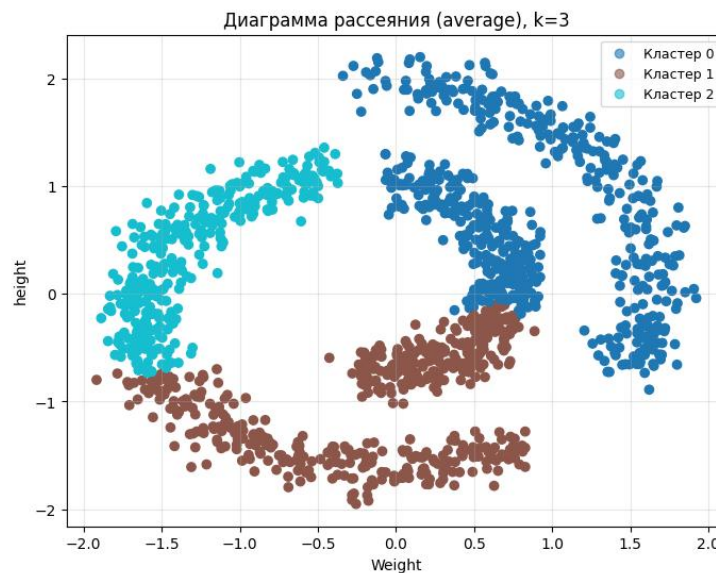


Рисунок 4.2.2 - Диаграмма рассеяния (average), k=3.

Алгоритм разрезал спирали поперек, кластеры не соответствуют естественной форме данных. Average linkage объединяет кластеры на основе среднего евклидова расстояния между всеми парами точек, что заставляет алгоритм стремиться к компактным сферическим группам и разрезать изогнутые спиральные структуры поперек, игнорируя их естественную связность.

Метод ward:

```
hc = AgglomerativeClustering(n_clusters=3, linkage='ward')
labels = hc.fit_predict(X_role_scaled_tf)

plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_role_scaled_tf.iloc[:, 0], X_role_scaled_tf.iloc[:, 1],
c=labels, cmap='tab10')

handles, _ = scatter.legend_elements(prop="colors", alpha=0.6)
labels_legend = [f"Кластер {i}" for i in range(3)]
plt.legend(handles, labels_legend, fontsize=9, loc='best')

plt.title(f'Диаграмма рассеяния(ward), k=3')
plt.xlabel('Weight')
plt.ylabel('height')
plt.grid(alpha=0.3)
plt.show()
```

Результат работы показан на рис. 4.2.3.

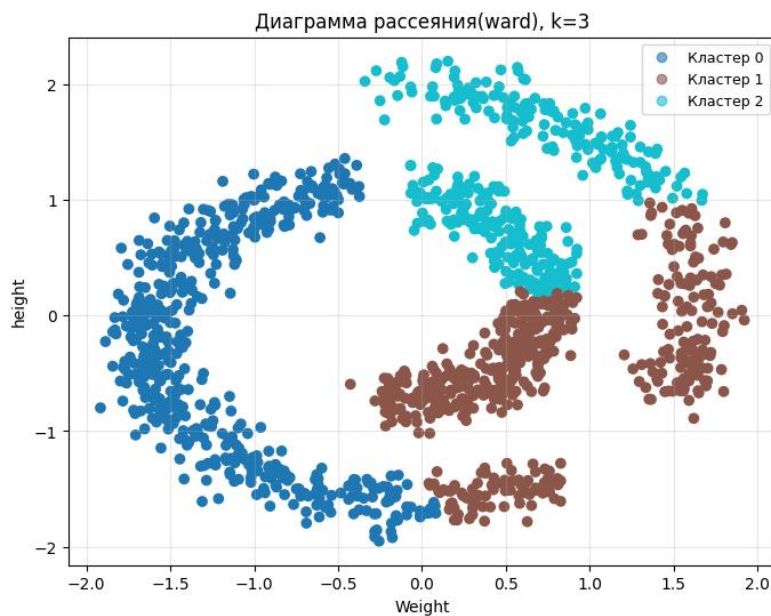


Рисунок 4.2.3 - Диаграмма рассеяния (ward), $k=3$.

Алгоритм разрезал спирали поперек, кластеры не соответствуют естественной форме данных. Ward linkage минимизирует внутрикластерную дисперсию, что заставляет алгоритм создавать компактные сферические кластеры примерно одинакового размера, разбивая изогнутые спиральные структуры на фрагменты и игнорируя их естественную связность.

Метод single лучшим образом разделил данные на кластеры.

4.3. Сравнение результатов кластеризации с результатами полученными в п.2 и п.3. Выводы о том, какой метод кластеризации подходит под каждый из наборов данных.

Для спиральных данных коэффициенты силуэта не рассчитывались, так как данная метрика основана на евклидовом расстоянии и предполагает компактную сферическую форму кластеров. Для нелинейных структур произвольной формы (спирали) значение силуэта может быть методически заниженным или не полностью отражать реальную корректность разбиения, поскольку точки внутри одной спиральной ветви могут быть значительно удалены друг от друга по прямой, хотя и принадлежат к одной естественной группе. Поэтому ключевыми критериями оценки выступали визуальная

интерпретация результатов, устойчивость структуры кластеров и соответствие геометрии данных.

Алгоритм K-means оказался неприменим для кластеризации спиральных данных, так как создает линейные границы между кластерами. Когда естественные группы данных переплетены и расположены близко друг к другу (как три спирали), их невозможно разделить прямыми линиями - алгоритм разрезает изогнутые структуры поперек, ошибочно объединяя точки из разных кластеров на основе евклидова расстояния.

Алгоритм DBSCAN показал качественные результаты кластеризации на данных со сложной нелинейной структурой. Алгоритм успешно выделил три спиральные ветви благодаря способности работать с кластерами произвольной формы и автоматически отсеивать шум. Однако результат сильно зависит от подбора гиперпараметров: некорректные значения `eps` и `min_samples` могут привести к избыточному дроблению кластеров или, наоборот, к их слиянию.

Иерархическая кластеризация также показала стабильные результаты и позволила исследовать структуру данных с помощью дендрограммы. При правильном выборе метода `linkage` удастся получить компактные и хорошо разделенные кластеры. Однако данный метод может быть менее эффективен на больших выборках из-за вычислительной сложности.

Выполним все задачи с файлом `lab4_blobs.csv`.

1. Загрузка данных

1.1. Загрузка данных.

Загрузим данные из файла через функцию `read_csv()` библиотеки `pandas`.

```
import pandas as pd
data_blobs = pd.read_csv('lab4_blobs.csv')
print(data_blobs.head())
```

Результат работы показан на рис. 1.1.

	Unnamed: 0	SkippedLessons	AverageGrade
0	0	130.0	1.1
1	1	54.0	0.5
2	2	58.0	1.0
3	3	107.0	0.9
4	4	42.0	1.2

Рисунок 1.1 - Демонстрация набора данных.

Набор данных содержит следующие столбцы:

- 1) Unnamed: 0 - индекс строки;
- 2) SkippedLessons — числовая характеристика, количество пропущенных занятий студентом;
- 3) AverageGrade — числовая характеристика, средний балл успеваемости студента.

Признаки SkippedLessons и AverageGrade будут использоваться для группировки объектов в кластеры на основе их близости в пространстве признаков. Кластеризация позволит выделить группы студентов со схожей успеваемостью и посещаемостью.

1.2. Проверка корректности загрузки.

Проверим данные на наличие пропущенных значений и корректность типов данных.

```
print(data_blobs.describe())  
print(data_blobs.info())
```

Результат работы показан на рис. 1.2.1 и рис. 1.2.2.

	Unnamed: 0	SkippedLessons	AverageGrade
count	1000.000000	1000.000000	1000.000000
mean	499.500000	77.865000	1.81410
std	288.819436	41.388909	1.21042
min	0.000000	13.000000	0.20000
25%	249.750000	45.000000	0.90000
50%	499.500000	61.000000	1.20000
75%	749.250000	123.000000	3.30000
max	999.000000	177.000000	4.30000

Рисунок 1.2.1 - Статистические характеристики набора данных.

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 1000 entries, 0 to 999  
Data columns (total 3 columns):  
#   Column             Non-Null Count  Dtype  
---  ---  
0   Unnamed: 0         1000 non-null   int64  
1   SkippedLessons     1000 non-null   float64  
2   AverageGrade       1000 non-null   float64  
dtypes: float64(2), int64(1)  
memory usage: 23.6 KB  
None
```

Рисунок 1.2.2 - Информация о наборе данных.

В наборе содержится 1000 наблюдений, пропущенные значения отсутствуют во всех столбцах. Признаки `SkippedLessons` и `AverageGrade` имеют вещественный тип (`float64`), что корректно для выполнения операций кластеризации.

`SkippedLessons` изменяется в пределах $[13; 177]$, среднее значение ~ 77.87 , стандартное отклонение ~ 41.39 .

`AverageGrade` изменяется в пределах $[0.2; 4.3]$, среднее значение ~ 1.81 , стандартное отклонение ~ 1.21 .

Признаки имеют разные диапазоны и стандартные отклонения (`SkippedLessons` варьируется значительно сильнее, чем `AverageGrade`). При использовании алгоритмов кластеризации, чувствительных к масштабу, признак с большим разбросом будет доминировать при расчете евклидова расстояния, поэтому нужно будет их масштабировать.

1.3. Построение диаграммы рассеяния набора данных.

Построим диаграмму рассеяния для визуальной оценки структуры данных и выявления потенциальных кластеров.

```
import matplotlib.pyplot as plt

plt.scatter(data_blobs['SkippedLessons'], data_blobs['AverageGrade'])
plt.xlabel('SkippedLessons')
plt.ylabel('AverageGrade')
plt.title('Диаграмма рассеяния признаков')
plt.legend()
plt.show()
```

Результат работы показан на рис. 1.3.

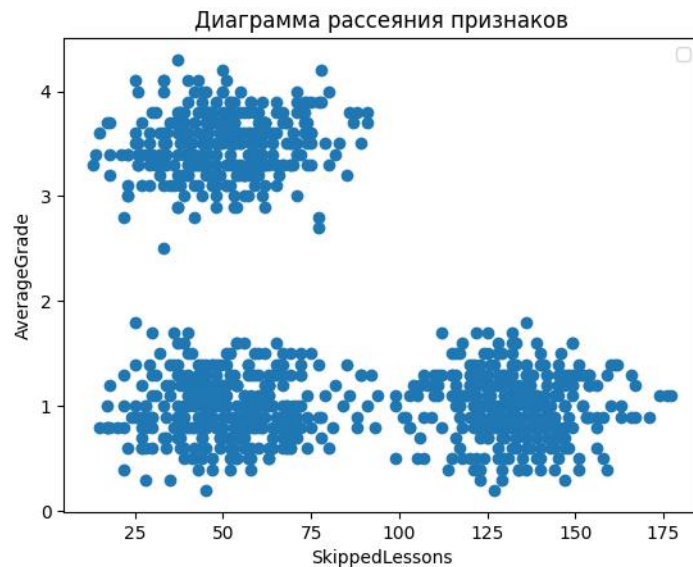


Рисунок 1.3 - Диаграмма рассеяния признаков.

На диаграмме рассеяния признаки `SkippedLessons` и `AverageGrade` образуют три компактные группы (облака точек), расположенные в разных областях пространства признаков:

- 1) Верхний левый кластер: студенты с высоким средним баллом ($\text{AverageGrade} \sim 2.5-4.0$) и низким/средним количеством пропусков ($\text{SkippedLessons} \sim 5-100$);
- 2) Нижний левый кластер: студенты с низким средним баллом ($\text{AverageGrade} \sim 0.5-2$) и низким/средним количеством пропусков ($\text{SkippedLessons} \sim 5-100$);
- 3) Нижний правый кластер: студенты с низким средним баллом ($\text{AverageGrade} \sim 0.5-1.5$) и высоким количеством пропусков ($\text{SkippedLessons} \sim 100-175$).

Плотность распределения точек внутри каждого кластера относительно равномерна. Такая структура данных указывает на наличие трех естественных групп студентов с различными паттернами успеваемости и посещаемости

1.4. Подготовка набора данных.

Проверим распределение данных и наличие выбросы.

```
data_blobs[['SkippedLessons', 'AverageGrade']].hist(bins=30)
plt.show()
```

```
plt.subplot(1,2,1)
plt.boxplot(data_blobs['SkippedLessons'])
plt.title('SkippedLessons')

plt.subplot(1,2,2)
plt.boxplot(data_blobs['AverageGrade'])
plt.title('AverageGrade')

plt.tight_layout()
plt.show()
```

Результат работы показан на рис. 1.4.1 и рис. 1.4.2

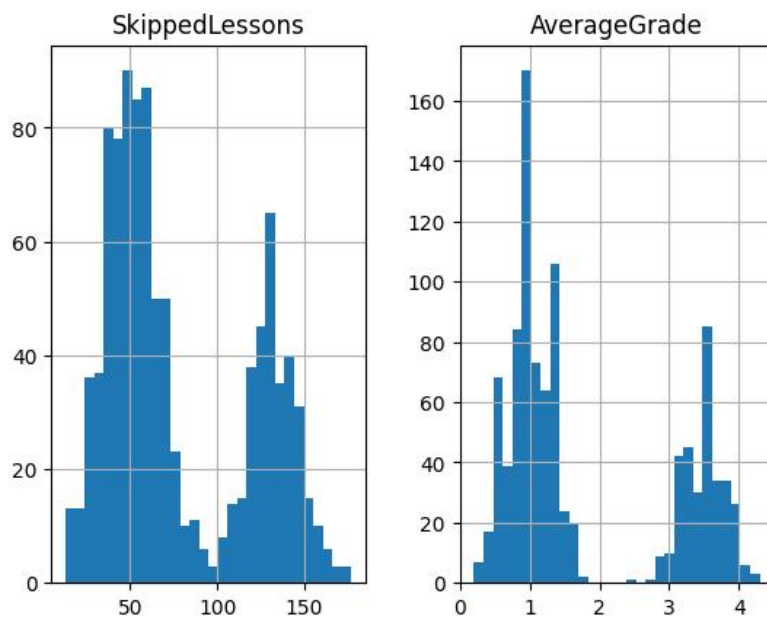


Рисунок 1.4.1 - Распределение признаков SkippedLessons и AverageGrade.

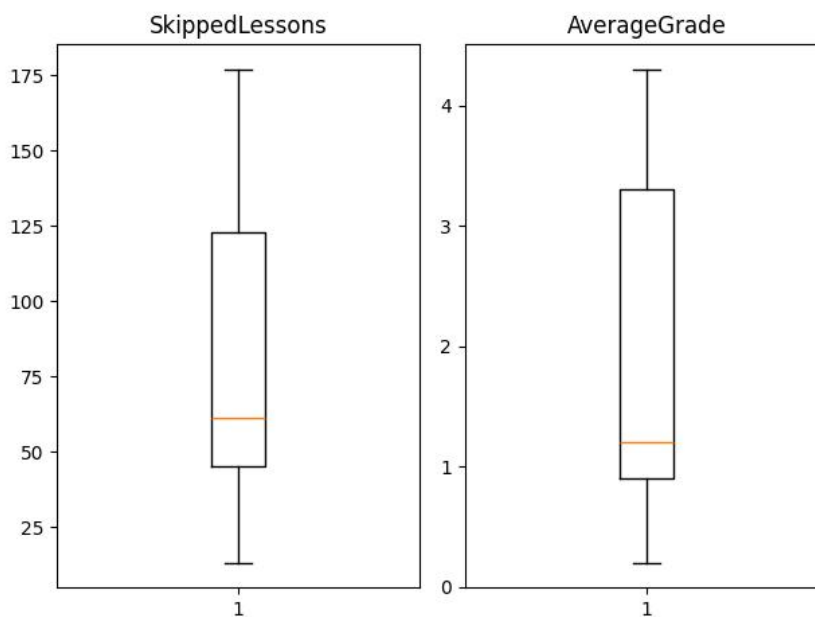


Рисунок 1.4.2 - Выбросы признаков SkippedLessons и AverageGrade.

Распределение SkippedLessons имеет два выраженных пика (около 50 и 130-140), значения находятся в диапазоне от 13 до 177 - такая форма распределения говорит о том, что в данных предположительно есть две группы объектов: студенты с низким/средним количеством пропусков и студенты с высоким количеством пропусков.

Распределение AverageGrade также имеет несколько пиков (около 1.0 и 3.5-4.0), значения находятся в диапазоне от 0.2 до 4.3 - это указывает на наличие различных групп успеваемости: студенты с низкой успеваемостью и студенты с высокой успеваемостью.

Признаки не имеют выбросов, поэтому можно использовать стандартизацию StandardScaler - она сохранит форму распределения данных и приведет признаки к единому масштабу с нулевым средним и единичной дисперсией.

```
from sklearn.preprocessing import StandardScaler

X_blobs = data_blobs[['SkippedLessons', 'AverageGrade']]
print(X_blobs[:5])

scaler = StandardScaler()
X_blobs_scaled = scaler.fit_transform(X_blobs)
X_blobs_scaled_tf = pd.DataFrame(X_blobs_scaled, columns=['SkippedLessons',
'AverageGrade'])
print('\n', X_blobs_scaled_tf.head())
```

Результат работы показан на рис. 1.4.3.

	SkippedLessons	AverageGrade
0	1.260267	-0.590256
1	-0.576892	-1.086200
2	-0.480200	-0.672913
3	0.704285	-0.755570
4	-0.866970	-0.507598

Рисунок 1.4.3 - Стандартизированные результаты.

После применения StandardScaler признаки приведены к единому масштабу.

Проверим получившиеся результаты через статистические характеристики и гистограмму распределения.

```
print(X_blobs_scaled_tf.describe())

X_blobs_scaled_tf.hist(bins=30)
plt.show()
```

Результат работы показан на рис. 1.4.4 и рис. 1.4.5.

	SkippedLessons	AverageGrade
count	1.000000e+03	1.000000e+03
mean	9.947598e-17	-6.394885e-17
std	1.000500e+00	1.000500e+00
min	-1.567991e+00	-1.334171e+00
25%	-7.944506e-01	-7.555704e-01
50%	-4.076802e-01	-5.075985e-01
75%	1.091055e+00	1.228205e+00
max	2.396405e+00	2.054778e+00

Рисунок 1.4.4 - Статистические характеристики стандартизированных данных.

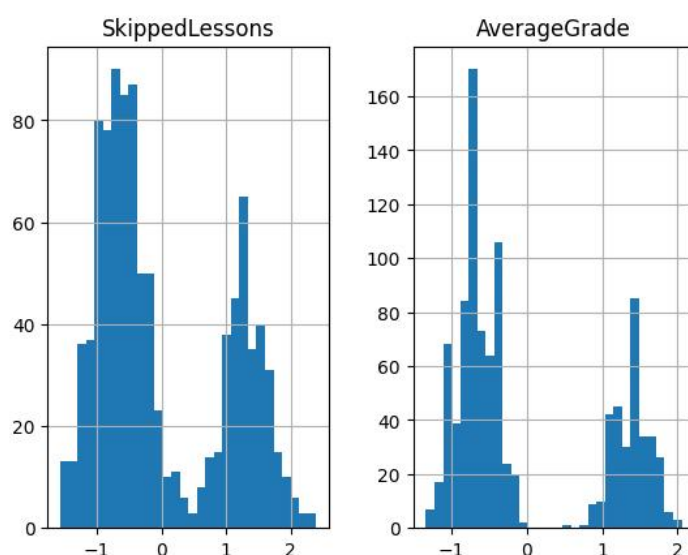


Рисунок 1.4.5 - Распределение стандартизированных признаков SkippedLessons и AverageGrade.

Средние значения обоих признаков близко к 0, стандартное отклонение равно ~ 1 для каждого признака, диапазон значений у SkippedLessons варьируется в пределах $[-1.8; 2.5]$, AverageGrade - в пределах $[-1.5; 2.3]$. Форма распределения сохранилась.

2. K-Means

2.1. Проведение исследования оптимального количества кластеров методов локтя.

Исследуем оптимальное количество кластеров для метода логтя.

```
from sklearn.cluster import KMeans

inertia = []

for k in range(1, 20):
    model = KMeans(n_clusters=k, random_state=42)
    model.fit(X_blobs_scaled_tf)
    inertia.append(model.inertia_)

plt.plot(range(1, 20), inertia, marker='o')
plt.title("Метод локтя")
plt.xlabel("k")
plt.xticks(range(1, 20))
plt.ylabel("Инерция")
plt.show()
```

Результат работы показан на рис. 2.1.

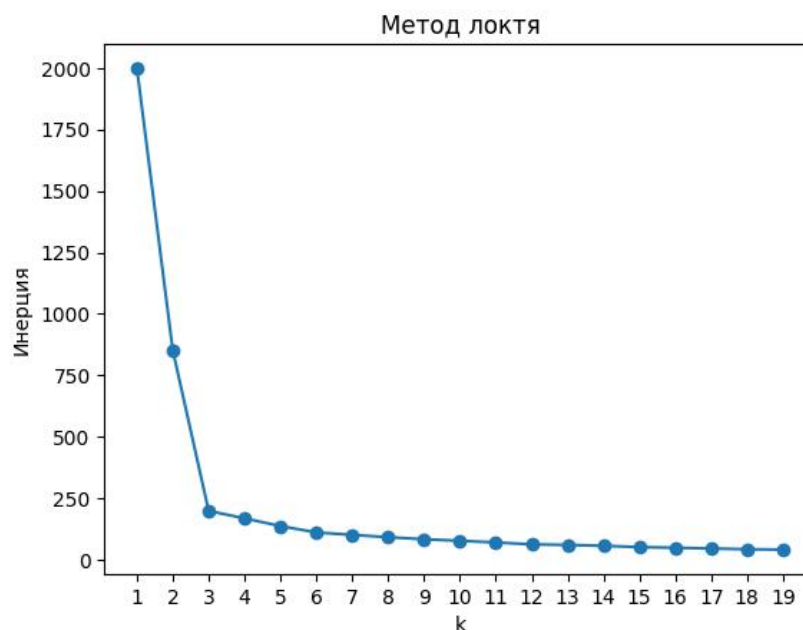


Рисунок 2.1 - Метод логтя.

На графике видно, что при k от 1 до 3 инерция резко падает - это значит, что добавление кластеров сильно улучшает результат. После $k = 3$ кривая изгибается, и снижение замедляется - новые кластеры дают все меньший эффект. При $k > 4$ инерция плавно уменьшается и приближается к нулю, что говорит о неэффективности дальнейшего увеличения количества кластеров. Оптимальное количество кластеров $k = 3$, так как в этой точке наблюдается наиболее выраженный перегиб графика.

2.2. Проведение исследования оптимального количества кластеров методом силуэта.

Исследуем оптимальное количество кластеров для метода силуэта.

```
from sklearn.metrics import silhouette_score

sil_scores = []

for k in range(2, 20):
    model = KMeans(n_clusters=k, random_state=42)
    labels = model.fit_predict(X_blobs_scaled_tf)
    sil_scores.append(silhouette_score(X_blobs_scaled_tf, labels))

plt.plot(range(2, 20), sil_scores, marker='o')
plt.title("Метод силуэта")
plt.xlabel("k")
plt.xticks(range(2, 20))
plt.ylabel("Коэффициент силуэта")
plt.show()
```

Результат работы показан на рис. 2.2.

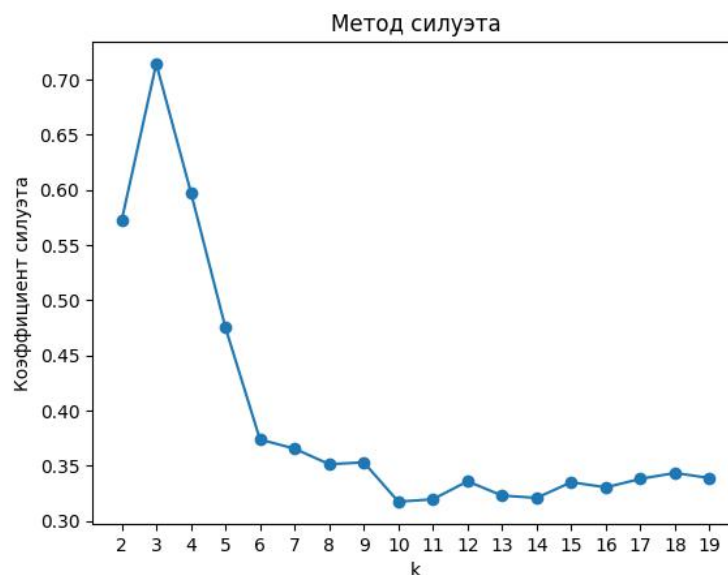


Рисунок 2.2 - Метод силуэта.

На графике представлен средний коэффициент силуэта для разного количества кластеров k . Максимальное значение ~0.72 достигается при $k = 3$ - это говорит о высоком качестве кластеризации. После $k = 3$ коэффициент силуэта резко снижается, постепенно выходя на плато ~0.35.

Оптимальное количество кластеров - $k = 3$. Это значение подтверждается обоими методами и согласуется с визуальным анализом данных.

2.3. Проведение кластеризации алгоритмом K-means, с выбранным оптимальным количеством кластеров.

Проведем кластеризацию алгоритмом K-means с количеством кластеров $k = 3$.

```
k_opt = 3
kmeans = KMeans(n_clusters=k_opt, random_state=42)
labels = kmeans.fit_predict(X_blobs_scaled_tf)
centroids = kmeans.cluster_centers_
```

Будем рассматривать 1 модель кластеризации K-means.

2.4. Построение диаграммы рассеяния результатов кластеризации с выделением разным цветом разных кластеров.

Построим диаграммы рассеяния результатов.

```
fig, axes = plt.subplots(figsize=(15, 10))
scatter = plt.scatter(X_blobs_scaled_tf['SkippedLessons'],
X_blobs_scaled_tf['AverageGrade'], c=labels)
centroid_marker = plt.scatter(centroids[:, 0], centroids[:, 1], c='red',
marker='X', s=200, edgecolors='black', label='Центроиды', cmap='tab10')
handles, _ = scatter.legend_elements(prop="colors", alpha=0.6)
cluster_legend = [f"Кластер {i}" for i in range(k_opt)]
plt.title(f'K-Means, k={k_opt}')
plt.xlabel('SkippedLessons')
plt.ylabel('AverageGrade')
plt.legend(handles=[*handles, centroid_marker], labels=[*cluster_legend,
'Центроиды'], fontsize=9, loc='best')
plt.tight_layout()
plt.show()
```

Результат работы показан на рис. 2.4.

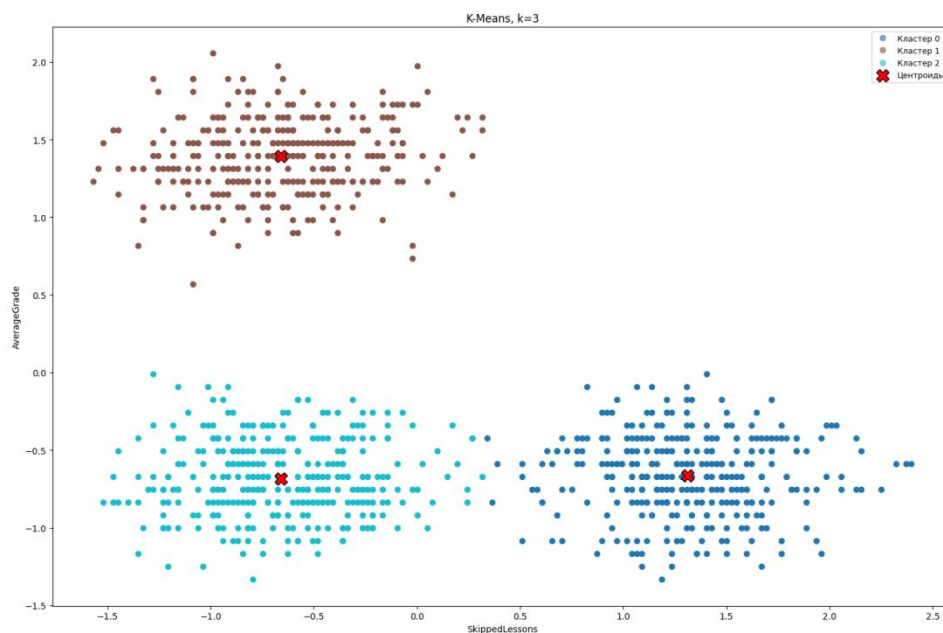


Рисунок 2.4 - Диаграммы рассеяния результатов кластеризации.

На диаграмме рассеяния видно, что K-means успешно выделил три естественных кластера в данных. Алгоритм корректно разделил группы студентов (подробно они описаны в п. 1.4).

В отличие от данных со спиральной структурой, K-means показал отличный результат на данном наборе данных, так как кластеры имеют компактную сферическую форму и хорошо разделены в пространстве признаков.

2.5. Построение диаграммы Вороного для результатов кластеризации..

Построим диаграммы Вороного для результатов кластеризации с классами $k = 3$.

```
from scipy.spatial import Voronoi, voronoi_plot_2d
import numpy as np

vor = Voronoi(centroids)
voronoi_plot_2d(vor)
scatter = plt.scatter(X_blobs_scaled[:, 0], X_blobs_scaled[:, 1], c=labels,
                      cmap='tab10')
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', marker='X', s=200,
            edgecolors='black', zorder=5)
plt.title(f'K-Means, k={k_opt}\nДиаграмма Вороного')
plt.xlabel('SkippedLessons')
plt.ylabel('AverageGrade')
plt.xlim(-2, 2.5)
plt.ylim(-1.5, 2.3)
plt.grid(True, alpha=0.3)
handles, _ = scatter.legend_elements(prop="colors", alpha=0.6)
cluster_legend = [f"Кластер {i}" for i in range(k_opt)]
plt.legend(handles=[*handles, centroid_marker], labels=[*cluster_legend,
'Центроиды'], fontsize=9, loc='best')
plt.show()
```

Результат работы показан на рис. 2.5.

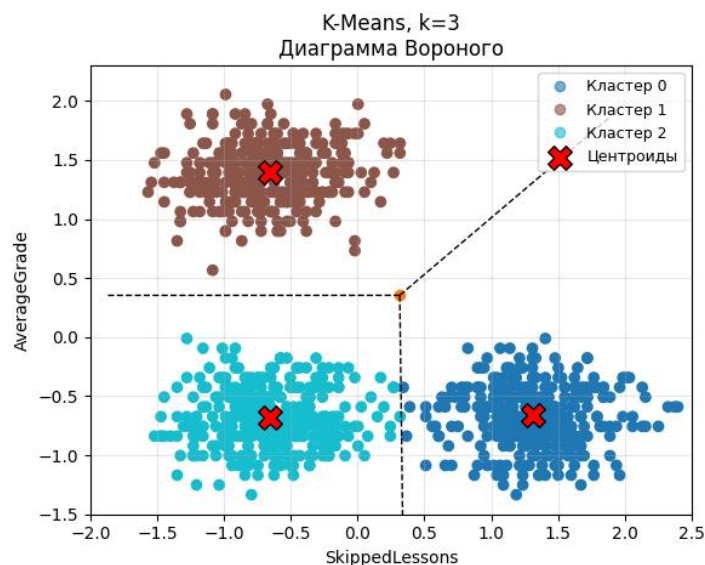


Рисунок 2.5 - Диаграмма Вороного для K-means при $k=3$.

При $k = 3$ ячейки компактно охватывают три естественные группы студентов, а границы проходят в областях с минимальной плотностью точек, что подтверждает корректность разделения.

2.6. Выводы о разделении кластеров и успешности применения кластеризации K-means к набору данных.

Построим диаграммы box-plot для каждого признака для K-means с количеством кластеров $k = 3$.

```
import seaborn as sns

df_plot = X_blobs_scaled_tf.copy()
df_plot['cluster'] = labels
df_plot['cluster'] = df_plot['cluster'].astype(str)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.boxplot(x='cluster', y='SkippedLessons', data=df_plot, ax=axes[0])
axes[0].set_title('SkippedLessons по кластерам')
axes[0].set_xlabel('Кластер')
axes[0].set_ylabel('SkippedLessons (стандартизированный)')
axes[0].grid(alpha=0.3, axis='y')

sns.boxplot(x='cluster', y='AverageGrade', data=df_plot, ax=axes[1])
axes[1].set_title('AverageGrade по кластерам')
axes[1].set_xlabel('Кластер')
axes[1].set_ylabel('AverageGrade (стандартизированный)')
axes[1].grid(alpha=0.3, axis='y')

plt.tight_layout()
plt.show()
```

Результат работы показан на рис. 2.6.

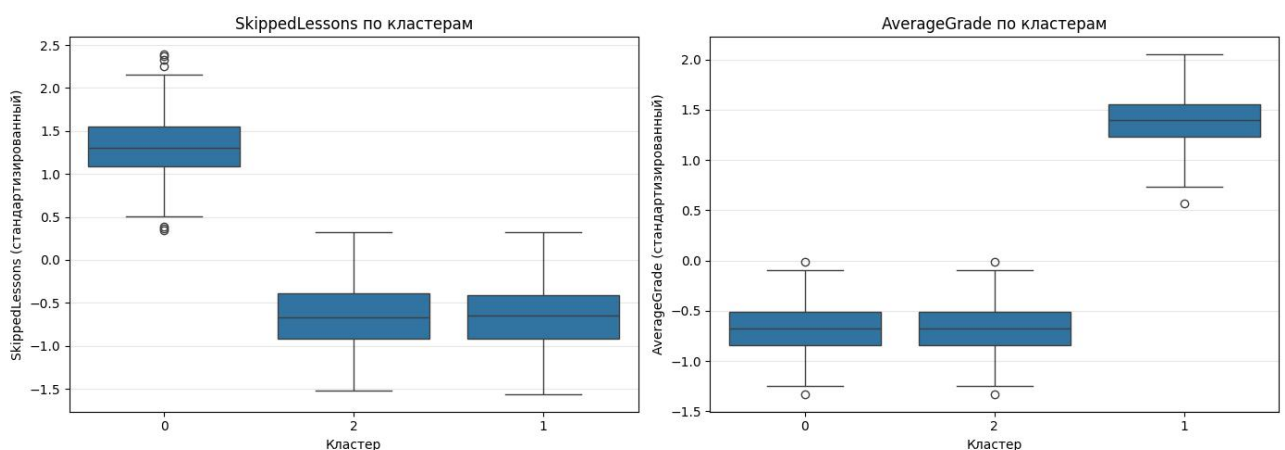


Рисунок 2.6 - Вох-plot для K-means кластеризации, $k=3$.

По признаку SkippedLessons кластер 0 существенно отличается от остальных: его значения смещены в положительную область (медиана ~ 1.3), тогда как кластеры 1 и 2 расположены в отрицательной (~ -0.6). По признаку AverageGrade кластер 1 отделен от остальных: его медиана значительно выше (~ 1.4). Кластеры 0 и 2 имеют схожие распределения по этому признаку.

Таким образом, ни один признак по отдельности не разделяет все три кластера, но их комбинация обеспечивает четкое разделение: кластер 0 выделяется по высокому значению SkippedLessons, кластер 1 - по высокому AverageGrade, а кластер 2 занимает уникальную позицию с низкими значениями обоих признаков.

При наличии компактных сферических кластеров алгоритм K-means работает эффективно, так как основывается на евклидовом расстоянии. Границы кластеров проходят корректно через области с низкой плотностью точек, что подтверждается высоким коэффициентом силуэта (~ 0.72) и четкой визуальной сепарацией на диаграмме Вороного.

2.7. Расчет количества точек, среднее, СКО, минимум и максимум. Сопоставление результатов с построенными графиками.

Рассчитаем количество точек, среднее, СКО, минимум и максимум для модели K-means $k=3$.

```
df_temp = X_blobs_scaled_tf.copy()
df_temp['cluster'] = labels
pd.set_option('display.width', 1000)

stats = df_temp.groupby('cluster').agg({
    'SkippedLessons': ['count', 'mean', 'std', 'min', 'max'],
    'AverageGrade': ['count', 'mean', 'std', 'min', 'max']
})

print(stats)
```

Результат работы показан на рис. 2.7.

cluster	SkippedLessons					AverageGrade				
	count	mean	std	min	max	count	mean	std	min	max
0	334	1.312594	0.362706	0.341687	2.396405	334	-0.662272	0.255168	-1.334171	-0.011655
1	326	-0.658458	0.377707	-1.567991	0.317514	326	1.392759	0.245243	0.566946	2.054778
2	340	-0.658086	0.367516	-1.519645	0.317514	340	-0.684825	0.245642	-1.334171	-0.011655

Рисунок 2.7 - Статистические характеристики признаков K-means для $k=3$.

По признаку `SkippedLessons` кластер 0 четко отделен от кластеров 1 и 2: имеет положительное среднее (1.31) и диапазон [0.34; 2.40], тогда как кластеры 1 и 2 имеют отрицательные средние (-0.66) и диапазоны [-1.57; 0.32] и [-1.52; 0.32] соответственно. Перекрытие отсутствует — максимум кластеров 1 и 2 (0.32) меньше минимума кластера 0 (0.34).

По признаку `AverageGrade` кластер 1 также четко отделен от кластеров 0 и 2: имеет положительное среднее (1.39) и диапазон [0.57; 2.05], тогда как кластеры 0 и 2 имеют отрицательные средние (-0.66 и -0.68) и диапазоны [-1.33; -0.01]. Перекрытие полностью отсутствует — минимум кластера 1 (0.57) выше максимумов кластеров 0 и 2 (-0.01).

Каждый признак четко разделяет один кластер от двух других, а комбинация признаков обеспечивает полное разделение всех трех групп без перекрытий. Небольшие стандартные отклонения (~0.25–0.38) указывают на компактность кластеров. Это подтверждает высокую эффективность K-means на данных с компактными сферическими кластерами и согласуется с высоким коэффициентом силуэта (~0.72).

3. DBSCAN

3.1. Подбор параметров алгоритма DBSCAN.

Найдем нужный диапазон для `eps` - для этого нужно построить графики `k`-расстояний, где `k` соответствует параметру `min_samples`. Для параметра `min_samples` буду рассматривать диапазон от 2 до 10.

```
from sklearn.neighbors import NearestNeighbors

for k in range(2, 11):
    neigh = NearestNeighbors(n_neighbors=k)
    neigh.fit(X_blobs_scaled_tf)

    distances, _ = neigh.kneighbors(X_blobs_scaled_tf)
    distances = np.sort(distances[:, -1])

    plt.plot(distances, label=f'k={k}')

plt.xlabel('Точки (отсортированные по расстоянию)')
plt.ylabel(f'Расстояние до k-го соседа')
plt.title('График k-расстояний для подбора eps')
plt.legend(loc='best', fontsize=9)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

Результат работы показан на рис. 3.1.1.

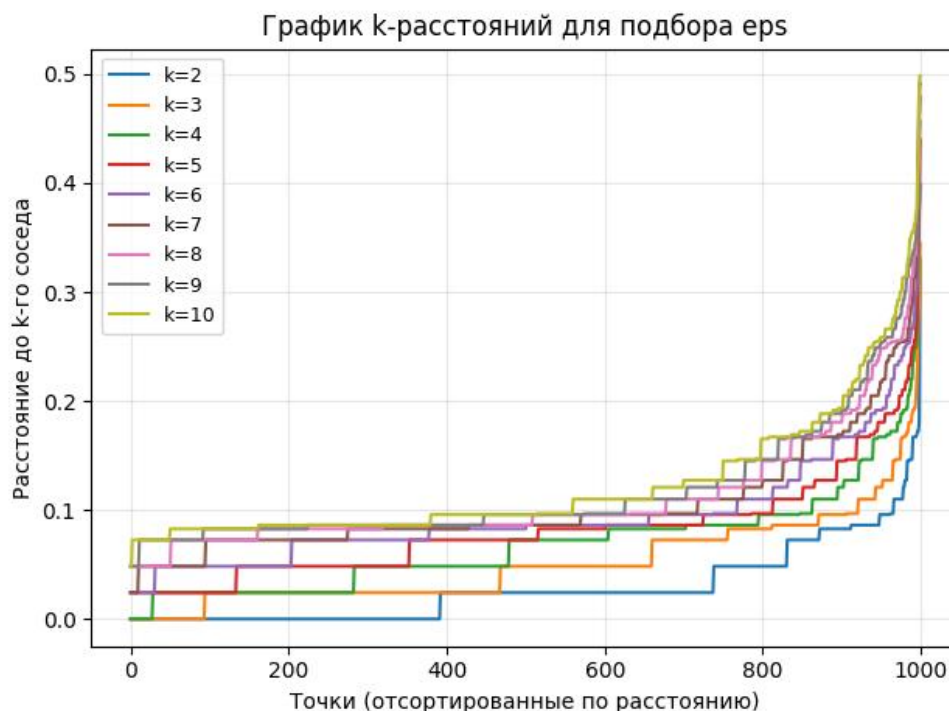


Рисунок 3.1.1 - График k-расстояний для подбора eps.

На графиках k-расстояний видно характерную ступенчатую форму, что обусловлено наличием трех компактных кластеров - внутри каждого кластера точки имеют схожую плотность и расстояния до k-го соседа одинаковы, но при переходе между кластерами плотность меняется и происходит скачок расстояний. Область локтя графиков лежит в диапазоне eps от 0.1 до 0.25 включительно, исследуем его.

Кластеризируем данные с помощью DBSCAN при разных гиперпараметрах и построим зависимость количества кластеров от min_samples (рассмотрим также диапазон 2-10) для разных значений eps (0.1-0.25).

```
from sklearn.cluster import DBSCAN

eps_values = np.arange(0.1, 0.26, 0.03)
results = {}
min_samples_values = list(range(2, 16))

for min_samp in min_samples_values:
    results[min_samp] = {}
    for eps in eps_values:
        db = DBSCAN(eps=eps, min_samples=min_samp)
        labels = db.fit_predict(X_blobs_scaled_tf)
```

```

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
noise = list(labels).count(-1)
results[min_samp][eps] = {'clusters': n_clusters, 'noise': noise}

fig, axes = plt.subplots(2, 3, figsize=(20, 10))
axes = axes.ravel()

for idx, eps in enumerate(eps_values):
    cluster_values = [results[min_s][eps]['clusters'] for min_s in
min_samples_values]

    axes[idx].plot(min_samples_values, cluster_values, marker='o', linewidth=2,
markersize=6)
    axes[idx].axhline(y=3, color='green', linestyle='--', linewidth=2,
label='Ожидаемое количество кластеров (3)')
    axes[idx].set_xlabel('min_samples', fontsize=11)
    axes[idx].set_ylabel('Количество кластеров', fontsize=11)
    axes[idx].set_title(f'eps = {eps:.2f}', fontsize=12, fontweight='bold')
    axes[idx].grid(alpha=0.3)
    axes[idx].set_xticks(min_samples_values)
    axes[idx].set_ylim(0, max(cluster_values) + 1)
    axes[idx].legend(fontsize=9)

plt.suptitle('Зависимость количества кластеров от min_samples для разных
значений eps', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

```

Результат работы показан на рис. 3.1.2.

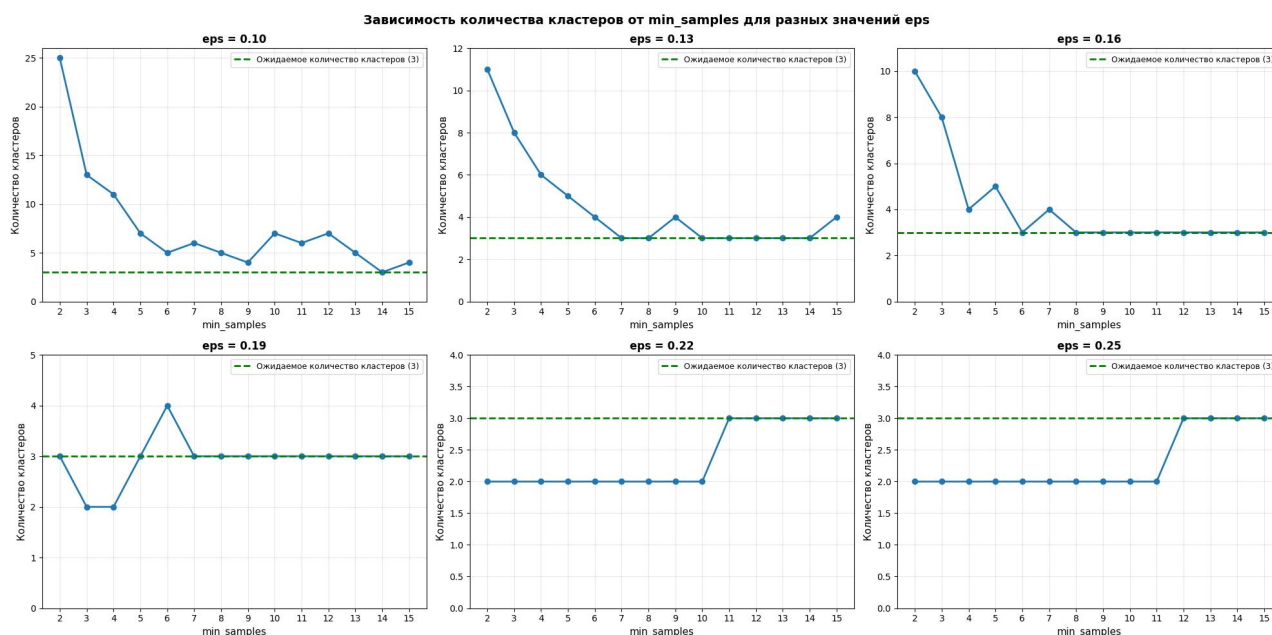


Рисунок 3.1.2 - Зависимость количества кластеров от min_samples для разных значений eps.

При $\text{eps} = 0.10-0.16$ алгоритм выделяет от 4 до 25 кластеров независимо от min_samples, что говорит об избыточном дроблении данных из-за слишком малого радиуса окрестности.

При $\text{eps} = 0.19$ достигается практически стабильное выделение ровно 3 кластеров в широком диапазоне min_samples (7-15), а также при $\text{min_samples} = 2$ и 5. Это показывает оптимальный баланс между чувствительностью к плотности и устойчивостью к шуму.

При $\text{eps} = 0.22-0.25$ при малых min_samples (2-10) алгоритм выделяет только 2 кластера, что говорит о слиянии естественных групп. Три кластера появляются лишь при $\text{min_samples} \geq 11-12$.

Для данного набора данных важно, чтобы алгоритм выделял осмысленные плотные области, а не случайные куски из нескольких точек. Параметр min_samples определяет порог, при котором группа точек считается ядром кластера: при слишком малых значениях (2-3) даже локальные плотности могут интерпретироваться как отдельные кластеры - это приводит к избыточному дроблению.

С другой стороны, чрезмерно большие значения min_samples могут игнорировать небольшие, но значимые группы. Для проверки устойчивости выбраны значения $\text{min_samples} = 5$ (охватывает весь спектр: от избыточного дробления к целевым трем кластерам и их объединению), 12 (промежуточное значение между нестабильной работой и стабилизацией), 14 (наиболее стабильное, обеспечивающее разделение на три кластера при всех eps) и 15 (крайнее значение). Такой выбор позволяет анализировать как переходные режимы, так и области стабильной работы алгоритма.

3.2. Построение диаграммы рассеяния результатов кластеризации с выделением разным цветом разных кластеров.

Построим диаграммы рассеяния для $\text{min_samples} = 5, 12, 14$ и 15 и $\text{eps}=0.1-0.26$.

```
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.ravel()

for i, eps in enumerate(eps_values):
    db = DBSCAN(eps=eps, min_samples=5)
    labels = db.fit_predict(X_blobs_scaled)

    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    n_noise = list(labels).count(-1)
```

```

scatter = axes[i].scatter(X_blobs_scaled[:, 0], X_blobs_scaled[:, 1],
c=labels, cmap='tab10')

axes[i].set_title(f'eps = {eps:.2f}\nКластеров: {n_clusters}, Шум: {n_noise}')
axes[i].grid(alpha=0.3)

handles, _ = scatter.legend_elements(prop="colors", alpha=0.6)

unique_labels = sorted(set(labels))
labels_legend = []
for label in unique_labels:
    if label == -1:
        labels_legend.append("Шум")
    else:
        labels_legend.append(f"Кластер {label}")

if len(labels_legend) <= 10:
    axes[i].legend(handles, labels_legend, fontsize=7, loc='best')
else:
    axes[i].legend([f"Всего кластеров: {n_clusters}"], fontsize=8,
loc='upper right')

plt.tight_layout()
plt.show()

```

Результат работы показан на рис. 3.2.1, рис. 3.2.2, рис. 3.2.3 и рис. 3.2.4.

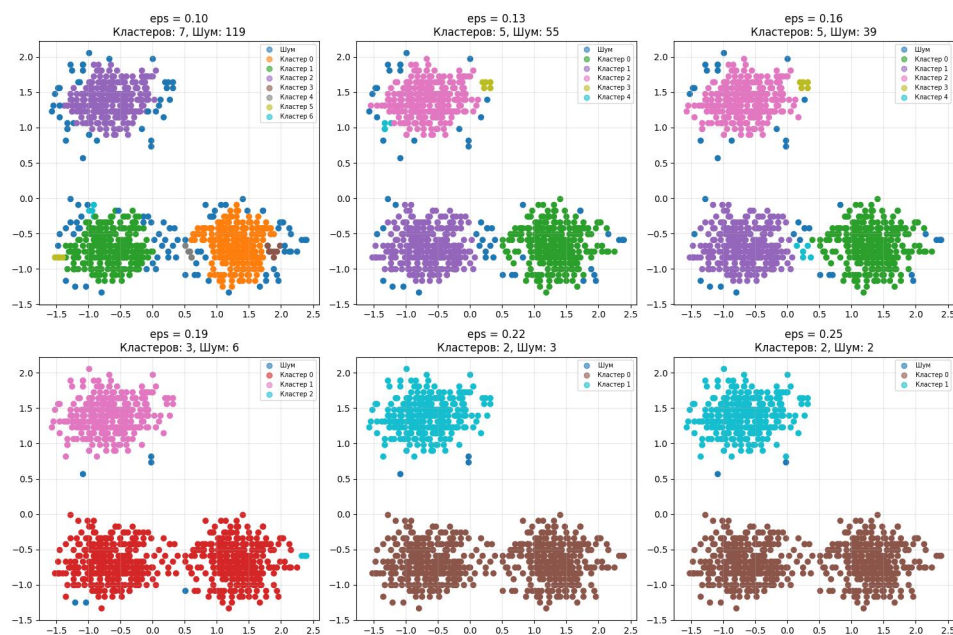


Рисунок 3.2.1 - Диаграммы рассеяния для разных eps и min_samples=5.

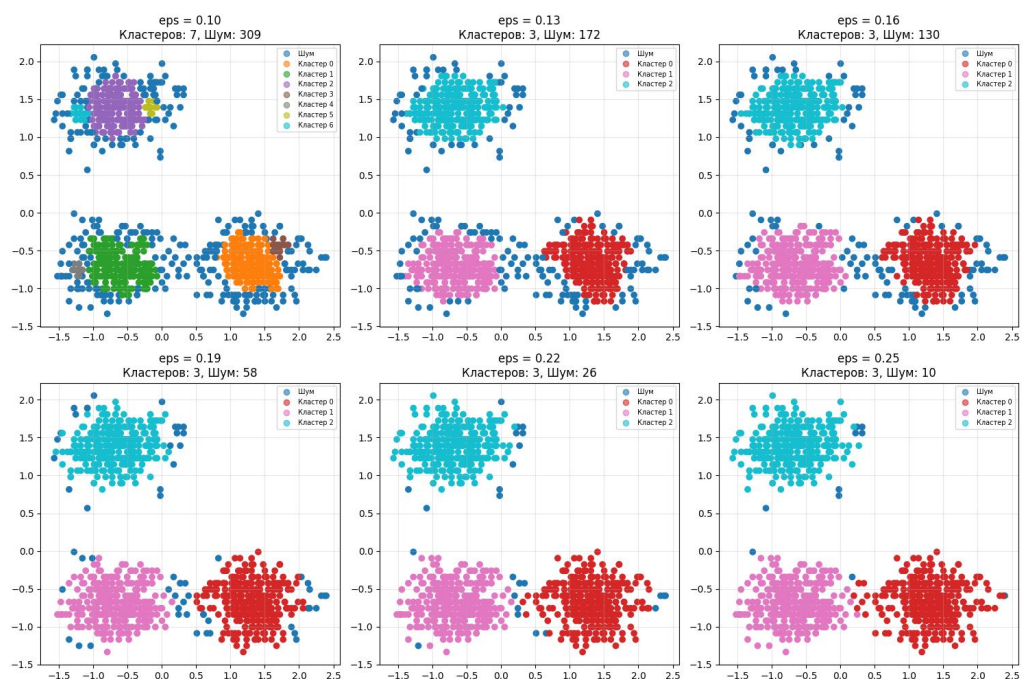


Рисунок 3.2.2 - Диаграммы рассеяния для разных eps и min_samples=12.

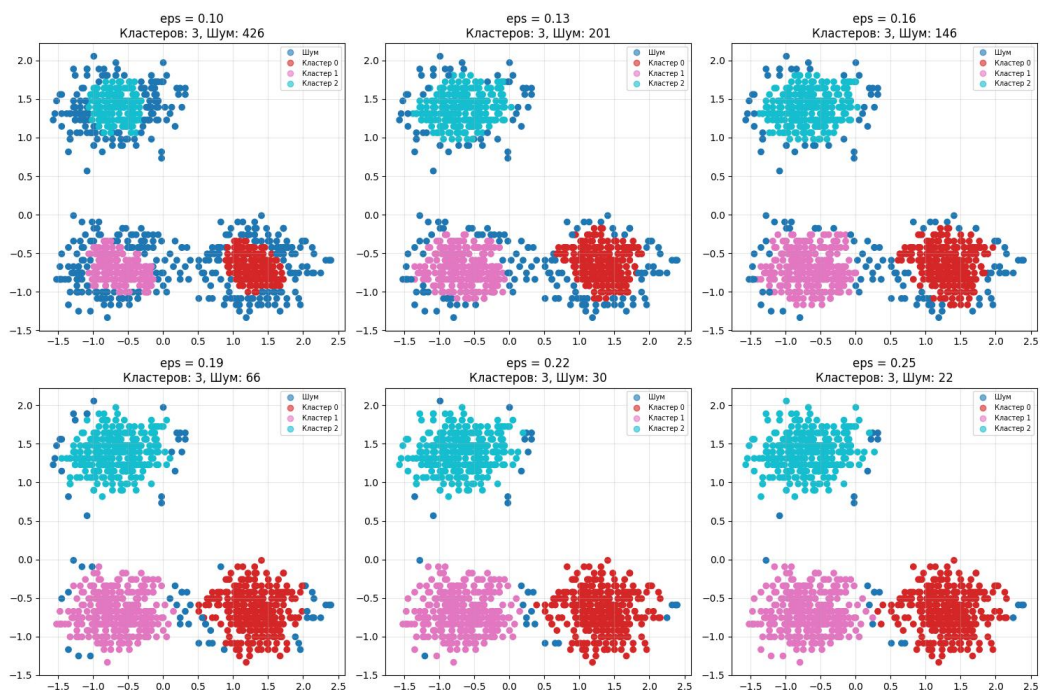


Рисунок 3.2.3 - Диаграммы рассеяния для разных eps и min_samples=14.

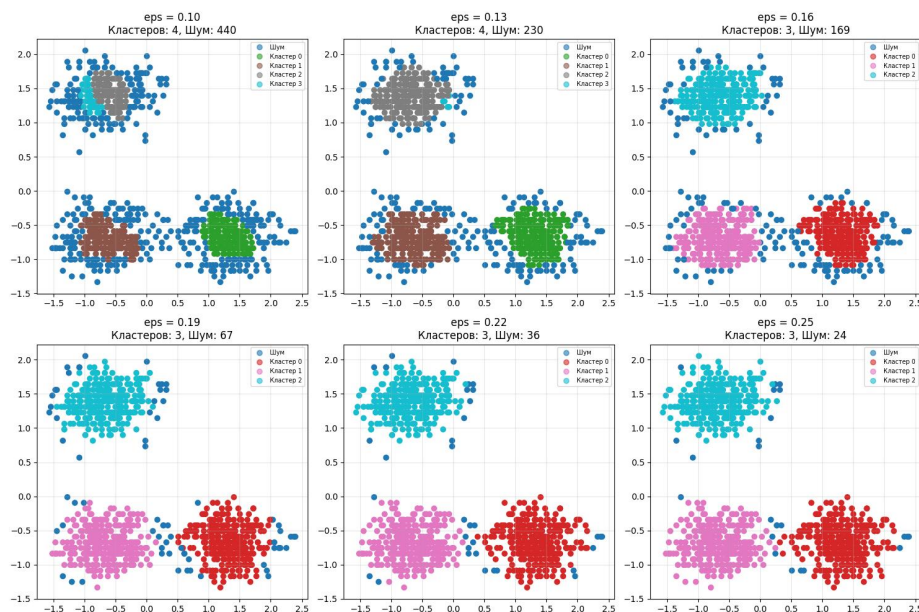


Рисунок 3.2.4 - Диаграммы рассеяния для разных ϵ и $\min_samples=15$.

При $\min_samples=5$ при $\epsilon=0.10-0.16$ наблюдается фрагментация данных на 5-7 кластеров с шумом 39-119 точек. При $\epsilon=0.19$ достигается 3 кластера (67 шумовых точек), но при $\epsilon=0.22-0.25$ происходит слияние до 2 кластеров. Малое $\min_samples$ приводит к нестабильности: алгоритм склонен либо к дроблению, либо к слиянию кластеров.

При $\min_samples=12$ при $\epsilon=0.10$ все еще наблюдается дробление (7 кластеров) с высоким шумом (309 точек). При $\epsilon=0.13-0.16$ кластеры начинают верно разделяться, но количество шумовых точек еще много (172 и 130 точек соответственно). При $\epsilon=0.19-0.25$ достигается стабильное выделение трех кластеров с минимальным шумом (10-58 точек).

При $\min_samples=14$ поведение схоже с $\min_samples=12$, но с большим уровнем шума. При $\epsilon=0.10-0.16$ выделяется 3 кластера (146-426 шумовых точек). При $\epsilon=0.19-0.25$ также выделяется три кластера, только с меньшим шумом в 22-66 точек.

При $\min_samples=15$ при $\epsilon=0.10-0.16$ выделяется 3-4 кластера с максимальным шумом (169-440 точек). При $\epsilon=0.19-0.25$ алгоритм стабильно выделяет три кластера с шумом 24-67 точек. Несмотря на повышенный уровень шума, структура кластеров остается устойчивой.

Все конфигурации при $\text{eps}=0.22-0.25$ обеспечивают выделение трех кластеров. Однако количество шума существенно различается: от 10 точек при $\text{min_samples}=12$ до 67 точек при $\text{min_samples}=15$.

3.3. Выводы об успешности кластеризации.

Эксперименты с DBSCAN показали, что алгоритм успешно справился с кластеризацией сферических данных.

При оптимальных параметрах ($\text{eps}=0.19-0.22$ и $\text{min_samples}=12-15$) алгоритм выделил 3 основных кластера с минимальным количеством шумовых точек (10-67).

DBSCAN эффективен для работы с данными, характеризующимися нелинейной структурой и плотностным распределением кластеров. Вместе с тем эксперименты подтвердили, что работа алгоритма сильно зависит от тщательного подбора параметров соседства (eps) и минимальной плотности (min_samples), поскольку их неправильный выбор приводит либо к чрезмерной фрагментации данных, либо к нежелательному объединению естественных групп.

4. Иерархическая кластеризация

4.1. Проведение иерархической кластеризацию, подобрав параметр linkage.

Построение дендрограммы.

Построим для всех методов linkage (single, complete, average, ward) дендрограммы.

```
from scipy.cluster.hierarchy import dendrogram, linkage

linkage_methods = ['single', 'complete', 'average', 'ward']

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.ravel()
for idx, method in enumerate(linkage_methods):
    try:
        Z = linkage(X_blobs_scaled_tf, method=method)
        dendro_data = dendrogram(Z, no_plot=True)

        dendrogram(Z, ax=axes[idx], truncate_mode='lastp', p=30)
        axes[idx].set_title(f'Метод linkage: {method}')
        axes[idx].set_xlabel('Индекс точки')
        axes[idx].set_ylabel('Расстояние')
        axes[idx].grid(alpha=0.3)
    except Exception as e:
```

```

        axes[idx].text(0.5, 0.5, f'Ошибка: \n{str(e)}', ha='center',
va='center', transform=axes[idx].transAxes)
        axes[idx].set_title(f'Метод: {method}')

plt.suptitle('Дендрограммы для разных методов linkage', fontsize=14,
fontWeight='bold')
plt.tight_layout()
plt.show()

```

Результат работы показан на рис. 4.1.

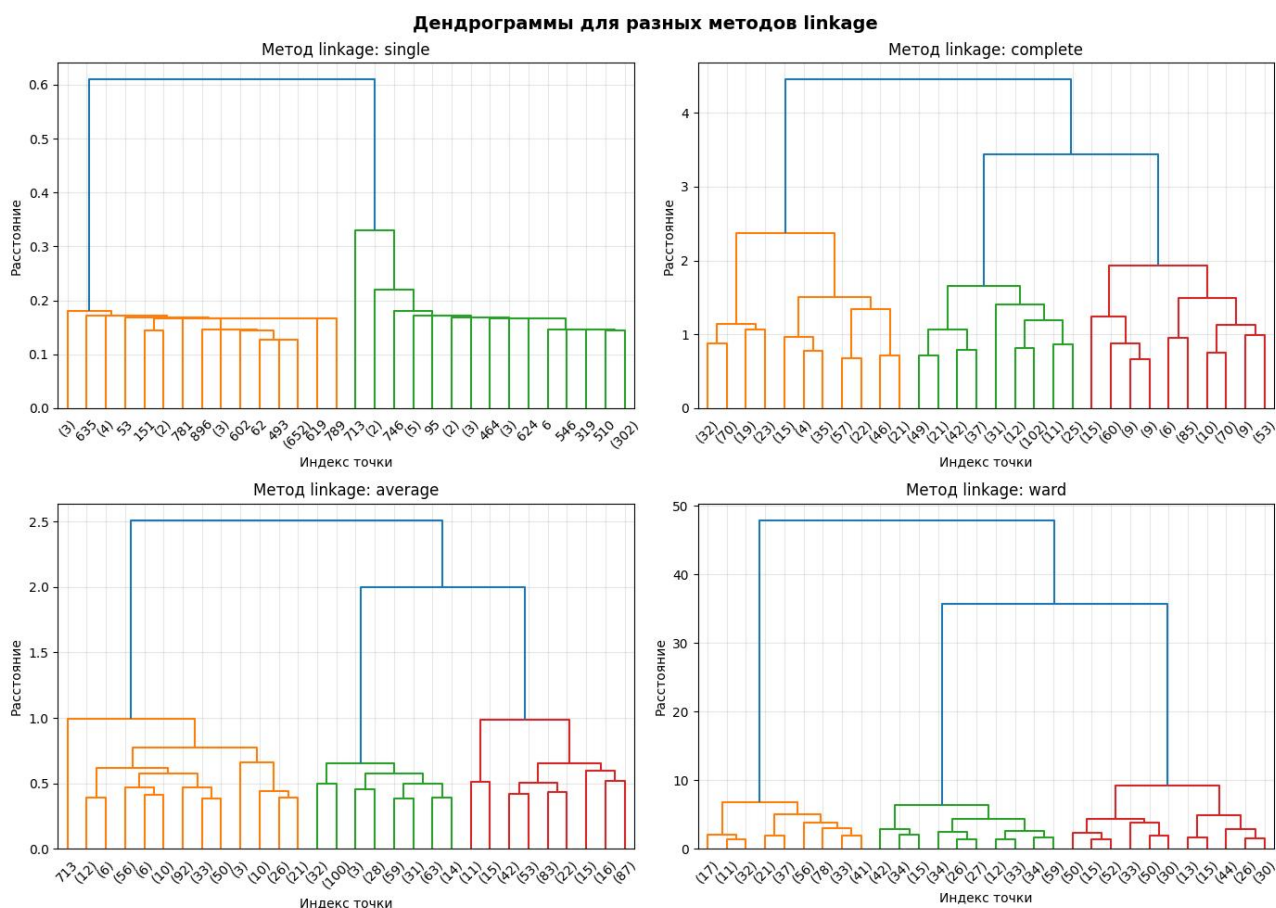


Рисунок 4.1 - Дендрограммы для разных методов linkage.

Метод single linkage показывает две выраженные ветви (оранжевая и зеленая), развивающиеся независимо на малых расстояниях (0.0–0.2) и сливающиеся только на резком скачке (>0.6). Такая структура указывает на то, что алгоритм выделяет только два кластера, что не соответствует ожидаемой структуре данных (три естественные группы).

Метод complete linkage выявляет три основные ветви (оранжевая, зеленая и красная), формирующиеся на расстояниях 1.0–2.0 и объединяющиеся на уровне

3.0–4.0. Метод создает более компактные кластеры, так как использует максимальное расстояние между точками.

Метод average linkage также показывает три четкие ветви (оранжевая, зеленая, красная), сливающиеся на расстоянии >2.0 .

Метод ward linkage демонстрирует максимальные расстояния слияния (>40 – 50), что отражает стремление алгоритма минимизировать внутрикластерную дисперсию.

Поскольку single выделил только два кластера вместо ожидаемых трех, данный метод не подходит. Для дальнейшего анализа и построения диаграмм рассеяния будут рассмотрены методы complete, average и ward, так как все они демонстрируют устойчивое выделение трех основных кластеров.

4.2. Построение диаграммы рассеяния результатов кластеризации с выделением разным цветом разных кластеров.

Построим три диаграммы рассеяния для методов complete, average и ward и рассчитаем для каждого метода значение коэффициента силуэта.

Метод complete:

```
hc = AgglomerativeClustering(n_clusters=3, linkage='complete')
labels = hc.fit_predict(X_blobs_scaled_tf)

plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_blobs_scaled_tf.iloc[:, 0], X_blobs_scaled_tf.iloc[:, 1], c=labels)

handles, _ = scatter.legend_elements(prop="colors", alpha=0.6)
labels_legend = [f"Кластер {i}" for i in range(3)]
plt.legend(handles, labels_legend, fontsize=9, loc='best')
plt.title(f'Диаграмма рассеяния (complete), k=3')
plt.xlabel('SkippedLessons')
plt.ylabel('AverageGrade')
plt.grid(alpha=0.3)
plt.show()
```

Результат работы показан на рис. 4.2.1.

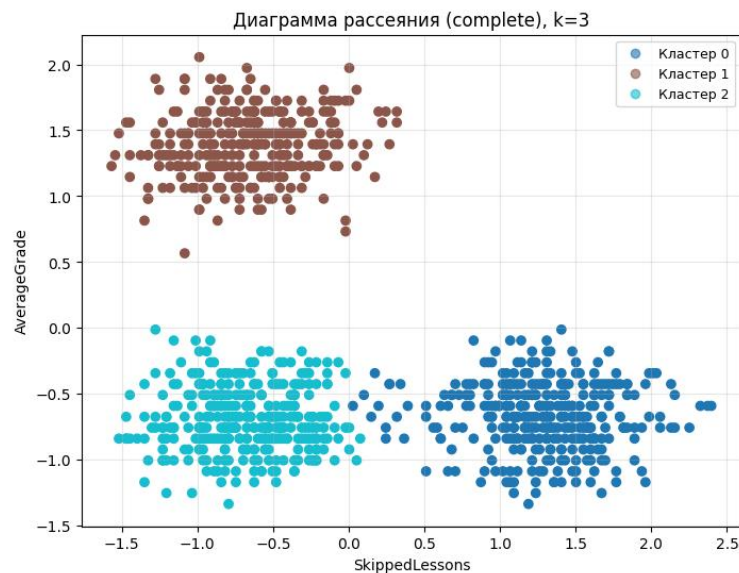


Рисунок 4.2.1 - Диаграмма рассеяния (complete), k=3.

Коэффициент силуэта равен 0.710, что указывает на хорошее качество кластеризации и хорошую разделимость групп.

Алгоритм корректно выделил три основных кластера, каждый окрашен в отдельный цвет. Метод complete linkage интерпретирует расстояние между кластерами через наиболее удалённые пары точек, поэтому формирует более строгие и «жёсткие» границы. Разделение между нижними кластерами проходит не по простой вертикальной линии, а по изогнутому контуру.

Метод average :

```
hc = AgglomerativeClustering(n_clusters=3, linkage='average')
labels = hc.fit_predict(X_blobs_scaled_tf)

plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_blobs_scaled_tf.iloc[:, 0], X_blobs_scaled_tf.iloc[:, 1], c=labels)

handles, _ = scatter.legend_elements(prop="colors", alpha=0.6)
labels_legend = [f"Кластер {i}" for i in range(3)]
plt.legend(handles, labels_legend, fontsize=9, loc='best')
plt.title(f'Диаграмма рассеяния (average), k=3')
plt.xlabel('SkippedLessons')
plt.ylabel('AverageGrade')
plt.grid(alpha=0.3)
plt.show()
```

Результат работы показан на рис. 4.2.2.

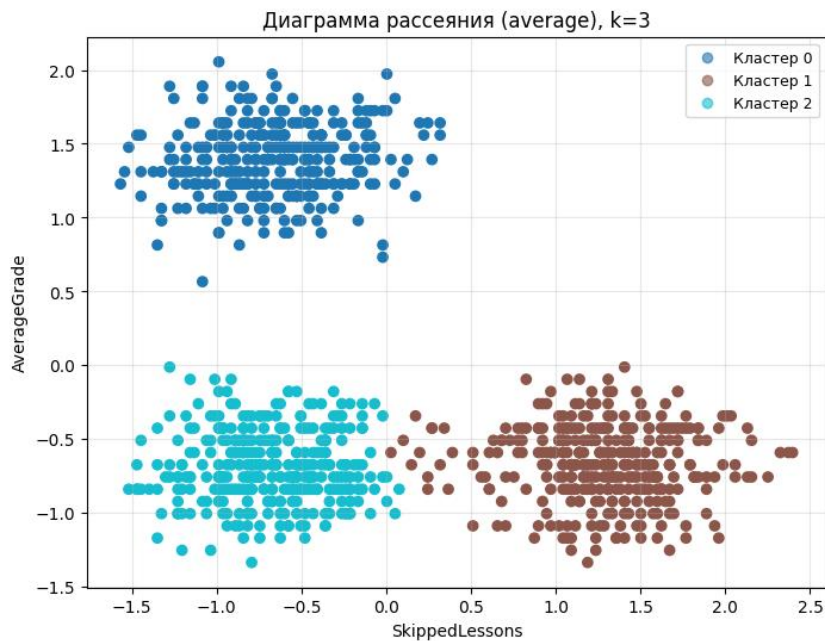


Рисунок 4.2.2 - Диаграмма рассеяния (average), k=3.

Коэффициент силуэта также равен 0.710, что указывает на хорошее качество кластеризации и хорошую разделимость групп.

Алгоритм корректно выделил три основных кластера. Метод average linkage формирует границы на основе средних попарных расстояний между точками, поэтому итоговое разбиение получилось практически идентичным методу complete.

Метод ward:

```
hc = AgglomerativeClustering(n_clusters=3, linkage='ward')
labels = hc.fit_predict(X_blobs_scaled_tf)

plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_blobs_scaled_tf.iloc[:, 0], X_blobs_scaled_tf.iloc[:, 1], c=labels)

handles, _ = scatter.legend_elements(prop="colors", alpha=0.6)
labels_legend = [f"Кластер {i}" for i in range(3)]
plt.legend(handles, labels_legend, fontsize=9, loc='best')

plt.title(f'Диаграмма рассеяния(ward), k=3')
plt.xlabel('SkippedLessons')
plt.ylabel('AverageGrade')
plt.grid(alpha=0.3)
plt.show()
```

Результат работы показан на рис. 4.2.3.

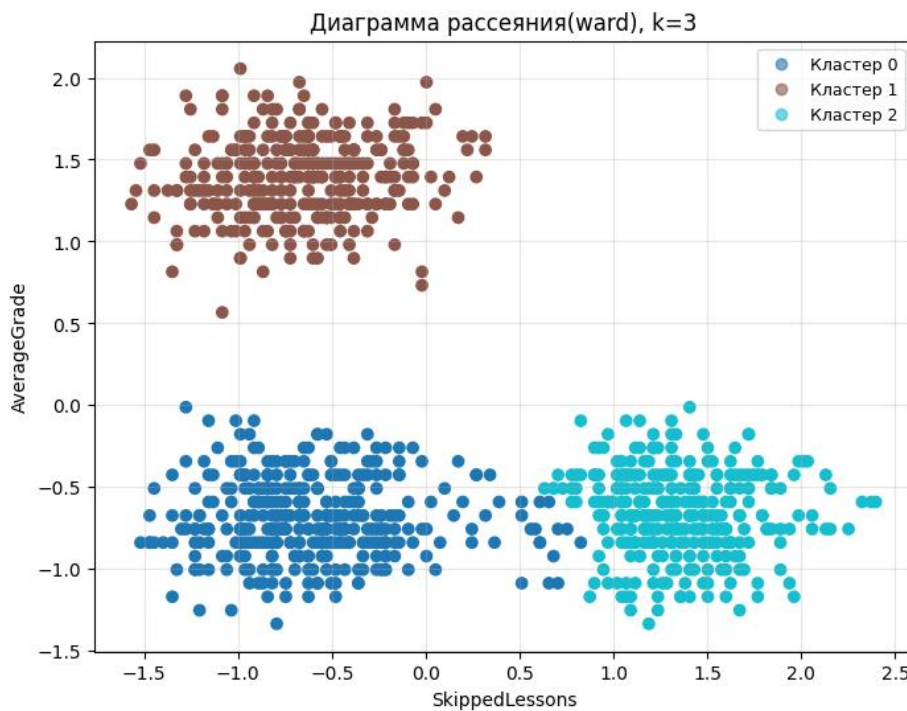


Рисунок 4.2.3 - Диаграмма рассеяния (ward), k=3.

Коэффициент силуэта равен 0.699 - это наименьшее значение среди трёх рассмотренных методов.

Алгоритм корректно выделил три основных кластера. Метод ward linkage по своему интерпретировал границы между кластерами: кластер 0 получился более протяжённым, потому что Ward минимизирует внутрикластерную дисперсию и стремится к более сбалансированным по вариативности группам. Поэтому часть пограничных точек, которые в других методах могли быть отнесены к кластеру 2 попала в кластер 0.

4.3. Сравнение результаты кластеризации с результатами полученными в п.2 и п.3. Выводы о том, какой метод кластеризации подходит под каждый из наборов данных.

Посчитаем коэффициенты силуэта для всех методов классификации (для метода DBSCAN был отдельно рассчитан наибольший коэффициент силуэта - он был достигнут при параметрах $\text{eps}=0.19$ и $\text{min_sample}=15$):

```
kmeans = KMeans(n_clusters=3, random_state=42)
labels_kmeans = kmeans.fit_predict(X_blobs_scaled_tf)
score_kmeans = silhouette_score(X_blobs_scaled_tf, labels_kmeans)
```



```

dbscan = DBSCAN(eps=0.19, min_samples=15)
labels_dbscan = dbscan.fit_predict(X_blobs_scaled_tf)

mask_noise = labels_dbscan != -1
X_no_noise = X_blobs_scaled_tf[mask_noise]
labels_no_noise = labels_dbscan[mask_noise]
score_dbscan = silhouette_score(X_no_noise, labels_no_noise)

hc = AgglomerativeClustering(n_clusters=3, linkage='complete')
labels_hc = hc.fit_predict(X_blobs_scaled_tf)
score_hc = silhouette_score(X_blobs_scaled_tf, labels_hc)

print(f"K-Means: {score_kmeans:.3f}")
print(f"DBSCAN: {score_dbscan:.3f} (без учёта {mask_noise.sum() -
len(X_blobs_scaled_tf)} шумовых точек)")
print(f"Hierarchical (complete/average): {score_hc:.3f}")

```

Результат работы показан на рис. 4.3.

```

K-Means: 0.714
DBSCAN: 0.742 (без учёта -67 шумовых точек)
Hierarchical (complete/average): 0.710

```

Рисунок 4.3 - Коэффициенты силуэта для всех методов кластеризации.

Сравнение методов K-means, DBSCAN и иерархической кластеризации показало, что все подходы корректно выделяют три основные группы на рассматриваемом наборе сферических данных, однако отличаются по устойчивости и чувствительности к параметрам.

Наибольшее значение силуэта показал DBSCAN, что говорит о наиболее четком разделении кластеров в оптимальной конфигурации параметров. Однако этот результат достигается только при корректном подборе `eps` и `min_samples`; при отклонении параметров алгоритм склонен либо к избыточному дроблению, либо к слиянию кластеров. K-means показал близкий по качеству и более стабильный результат, что хорошо согласуется с природой данных (компактные сферические кластеры). Иерархическая кластеризация также дала качественное и интерпретируемое разбиение, немного уступая по метрике силуэта, но предоставляя дополнительное преимущество в виде дендрограммы и наглядного анализа структуры данных.

Таким образом, для данного набора данных DBSCAN обеспечивает наилучшую делимость кластеров по метрике силуэта в оптимальных настройках, K-means является наиболее стабильным и практически удобным

методом, а иерархическая кластеризация - информативной альтернативой для структурного анализа данных.

Ссылка на полный код находится в Приложении.

Выводы

В ходе работы были исследованы и сравнены три метода кластеризации: K-means, DBSCAN и иерархическая кластеризация на двух типах данных - спиральных (roll) и сферических (blobs). Для каждого метода выполнен подбор параметров, построены визуализации кластеров и проведён сравнительный анализ качества.

Для набора roll (спиральные данные) метрика силуэта не использовалась как основная сравнительная оценка, поскольку она опирается на евклидовы расстояния и лучше отражает качество для компактных, выпуклых кластеров. В случае сложных нелинейных структур (переплетённые спирали) значение силуэта может быть методически заниженным или не полностью отражать реальную корректность разбиения. Поэтому ключевым критерием здесь выступали визуальная интерпретация, устойчивость структуры кластеров и соответствие геометрии данных. По результатам анализа K-means оказался неприменим к roll-данным: метод формирует линейные границы и «разрезает» спирали поперёк. DBSCAN показал наилучшее качество, корректно выделяя спиральные ветви за счёт способности находить кластеры произвольной формы и отделять шум. Иерархическая кластеризация также дала хорошие результаты (при удачном выборе linkage) и дополнительно позволила проанализировать структуру данных через дендрограмму, но менее удобна на больших объёмах из-за вычислительной сложности.

Для набора blobs (сферические данные) применение коэффициента силуэта является достаточно корректным и информативным, так как кластеры компактны. Получены следующие значения: K-means - 0.714, DBSCAN - 0.742 (без учёта шумовых точек), иерархическая кластеризация (complete/average) - 0.710. Результаты показывают, что все методы успешно выделяют три кластера, однако различаются по чувствительности к настройкам. Наибольшее значение силуэта получил DBSCAN, но только при оптимальных параметрах; при их изменении качество заметно падает (дробление или слияние

кластеров). K-means показал близкий к лучшему и наиболее стабильный результат, что делает его практичным выбором для сферических данных. Иерархическая кластеризация немного уступает по метрике, но остаётся качественным и интерпретируемым подходом.

Таким образом, итоговый выбор метода зависит от структуры данных: для нелинейных спиральных данных предпочтителен DBSCAN (или иерархический подход с подходящим linkage), а для компактных сферических данных наиболее универсальным и устойчивым решением является K-means, при этом DBSCAN может дать лучший результат по силуэту в узком диапазоне параметров.

ПРИЛОЖЕНИЕ

1. Ссылка на код со спиральными данными:

<https://colab.research.google.com/drive/1ywy2Ru7usG451wofnmZE3iRPiYj4a4Wp?usp=sharing>.

2. Ссылка на код со сферическими данными:

<https://colab.research.google.com/drive/1XAhkvdiQuKTZsDu63L9UdgDfv2XMQHCI?usp=sharing>.